

Main ERD

Authoring Platform Consolidation

Metadata

Authors: Anton Rogozhkin

ERD uPlan URL: [46647f17-caf2-4fce-a85a-4a51c275ab99](#)

Project Summary: Building a Content Management System (CMS) by consolidating functionality of Content-Entities (Typewriter), Unified-Content, Louvre and Camopress into a new service Auteur to handle authored content for Uber web properties, starting with UberEats SEO pages.

PRD:  Delivery Authoring User Requirements

uOwn Asset: [EATSCOM](#)

Date started: Jul 1, 2026

Value Proposition

We propose building a Content Management System (CMS) by consolidating functionality of Content-Entities (Typewriter), Unified-Content, and Louvre into a new Auteur service to handle authored content for Uber web properties, starting with UberEats SEO pages and then expanding to other pages.

The proposed backend functionality will reside in a new Auteur service since we will have a separate database, backend code (API, handlers, controllers, repositories, etc.). Authoring frontend will reside in Chameleon as separate views in Chameleon Fusion app.

The primary goal is to provide a single authoring flow for ubereats SEO pages with a performant backend, while keeping the option to scale and handle other content in future.

For content authors:

- A single authoring flow replaces the current patchwork of Typewriter, Unified-Content, Louvre, and Camopress, eliminating the need to juggle multiple tools and mental models.

- UberEats SEO pages become fully authorable — layout and conditional logic currently hardcoded in web-eats-v2 move into the CMS where authors can manage them directly.

For engineers:

- Fewer services to operate: consolidation eliminates cross-service coordination, reducing deployments, on-call rotations, and integration points that break.
- Change tracking: the system identifies which pages are affected when a block, layout override, or dataset changes.

For Uber:

- Faster time-to-market: an engineer creates a block once, and content authors can launch pages for hundreds of cities by editing a spreadsheet and clicking publish. Scaling content becomes a content team problem, not an engineering bottleneck.

Current challenges to achieving value proposition

Fragmented service architecture. Content authoring and delivery is spread across Typewriter (Content-Entities), Unified-Content, Louvre, Rolodex, and Camopress. Authors must navigate multiple tools to change a single page, and the mental model of "where content comes from" is hard to reason about.

High latency from sequential data fetching. The whole system is designed to retrieve data through multiple sequential queries across services. Each service performs its own chain of lookups (e.g., airport → territory → city → rides → ratings), blocking latency optimization. The existing uScript templates implement what are effectively SQL JOINS in an imperative template language, one lookup at a time.

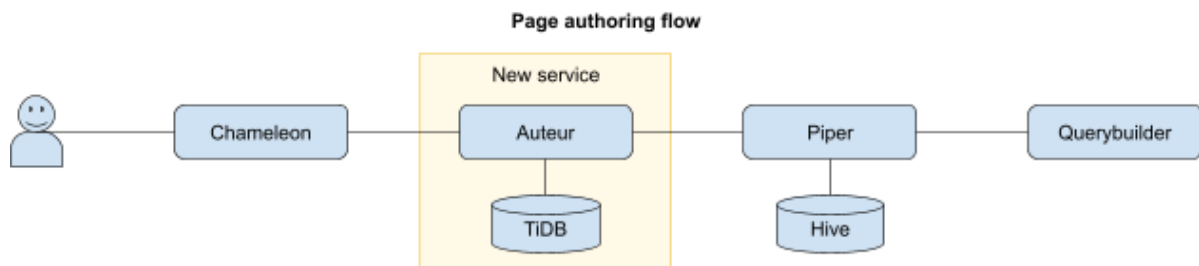
Uscript combines data retrieval with processing. In the current architecture, a block's uScript template fetches data and renders output in the same step. This coupling prevents the engine from collecting and parallelizing data queries across blocks.

Data distributed across services. Data living in multiple services makes it hard to track changes and understand impact. There is no single place to answer "what changed and which pages are affected?"

Non-Goals

- Not a website builder, just content management. The CMS will operate with visual components provided by EatsWebSEO service only, authors cannot create or modify how the content will look like.
- Not a replacement for existing functionality of Camopress, Chameleon Fusion, Typewriter, Content-Entities, Unified-Content, Guac, Louvre, or Rolodex for existing content. The new system is for new SEO page types; migration of existing pages is a separate, later effort.

Proposal (High level)



We propose consolidating the authoring capabilities of Typewriter, Unified-Content, Louvre, parts of Rolodex, and Camopress into a new backend service Auteur as a unified Content Management System. The system is organized around the concept: **URL → Layout → Blocks**.

Capabilities:

- **Hierarchical layout.** A base layout with block-level overrides at nested levels (country, vertical, slug). The layout is a three-dimensional override structure with axes: URL pattern, block position, and weight. At retrieval time, the engine filters overrides by URL pattern, groups by position, and selects the highest-weight override for each position.
- **Dynamic data mapping.** Blocks are not templates — they are data mappings from datasets to rendering models. Each block uses structured fields (queries, rules, content) instead of unstructured uScript. Because queries are declared separately from mapping logic, the engine can collect all queries across all blocks and execute them in parallel.

- **Dataset management.** Two types: editorial datasets (small, localized, manually maintained by authors) and dynamic datasets (large, derived from Hive queries, refreshed automatically by pipeline). Both share a uniform query interface from the block's perspective.
- **Custom localization.** Translations stored in TiDB alongside content, retrieved per page and locale. Direct integration with machine translation service for draft generation. Supports translation overrides.
- **Change management.** Draft versions and published change history. Authors create or modify content through Chameleon UI; the system combines changes into a changeSet, shows preview, and publishes.

Architecture overview:

- Database and backend code reside in a new backend service Auteur.
- Front-end authoring and dataset management reside in **Chameleon Fusion**.
- Frontend rendering resides in **EatsWebSEO** (Go SSR).
- **TiDB** as the primary database, QueryBuilder/Hive, and other Uber services.

Rendering flow: The engine fetches data in two phases: first it resolves the layout (computes the ordered list of blocks from layout overrides), then it executes all block queries in parallel. Pre-data rules (redirects, not-found) are evaluated before queries are issued; post-data rules are evaluated after.

Synchronization: A sync workflow pulls resolved page data from the Auteur Service and pushes it to Content Service (the serving cache). Content Service and DocStore remain on the serving path; TiDB unavailability means stale content, not an outage.

Affected Parties

- **Chameleon Fusion:** add layout-override and block editing UI, integrate with EatsWebSEO for preview.
- **Auteur Service:** new TiDB, APIs, handlers, controllers, and repositories for layout overrides, blocks, and datasets.

Engineering Risk

1. Migration risk — moving live pages without breaking them.

Millions of existing pages run on web-eats-v2 pulling data from EatsSEOGateway, other pages pull data from Content-Entities, Unified-Content, and Rolodex through Louvre.

Migrating them to blocks means rewriting every page's data logic and verifying the output matches exactly. A single mismatch in a field mapping can break SEO metadata or render wrong content.

Mitigation: Run old and new systems in parallel. For each URI, resolve through both paths and diff the output. Automated comparison catches regressions before cutover. Migrate one page type at a time (e.g., airport pages first).

2. TiDB as the primary store — unproven at this access pattern.

The current system uses DocStore and in-memory caches. Moving to TiDB for layout resolution, block fetching, and dataset lookups introduces a new dependency. If TiDB has a latency spike or goes down, pages could be affected.

Mitigation: Content Service (the caching layer) sits between TiDB and the rendering path. Pages are served from cache, not directly from TiDB. The sync worker populates the cache. TiDB unavailability means stale content, not an outage.

3. Mapping language complexity — building a mini-language is expensive.

The proposal replaces uScript with a new expression language (selectors, conditionals, each/as loops). Designing, implementing, testing, and documenting a custom language is a significant investment. Edge cases in expression evaluation become hard-to-debug production issues.

Mitigation: Keep the language minimal. Start with field selectors only (`airport.name`), add conditionals and loops only when a concrete page requires them. Consider using an existing expression engine (like CEL or JsonPath) instead of building from scratch.

Design

Namespaces

A namespace is defined by the domain plus the first URL path segment (e.g., `ubereats.com/region`).

Namespaces separate groups of content and control editing permissions.

Permissions:

- **Viewer** — all authenticated users.
- **Editor** — per namespace; can edit blocks and layout overrides within the namespace.

- **Admin** — can create and manage namespaces.

Hierarchical Layout

A layout is the computed ordered list of blocks for a given URL, resolved from layout-override records. The layout is hierarchical: a base set of overrides at the domain level, with additional overrides at each URL depth. Child pages inherit all blocks from parent levels and may override, add, or hide specific ones.

Layout resolution for **ubereats.com/region/san-francisco**

URL level	Blocks				
ubereats.com	Header	Hero	Categories	SEO Text	Footer
/region		Region Hero		Region SEO	
/san-francisco			SF Categories	~~hidden~~	SF Footer
	Header	Region Hero	SF Categories	~~hidden~~	SF Footer

Resolved page for /region/san-francisco: Header, Region Hero, SF Categories, (SEO Text hidden), SF Footer.

Each layout-override record maps a combination of domain, namespace, and URL pattern to a block ID, position, and weight. The engine filters layout overrides by URL, groups by position, and selects the highest-weight override.

Blocks

A block is a data mapping from datasets to a rendering model. Each block consists of three structured sections:

- **Queries:** named, decoupled data source declarations. Parameters can reference URL path segments, the current locale, or static values. All queries across all blocks on a page are collected and executed in parallel.
- **Rules:** an ordered list of conditions (redirect, not_found, null) evaluated top-down. The first matching rule determines the block result.
- **Content:** field mappings from query responses to a rendering model (e.g., **HeroBlock**, **CategoryNav**).

Authors edit blocks through Chameleon Fusion UI. On publish, the system validates the mapping against the rendering model and compiles it into a Protobuf representation stored in TiDB. At request time, only Protobuf decoding occurs.

Datasets

A dataset is a named collection of records that blocks query during page rendering. From the block's perspective, a dataset is a key-value store: given a primary key (or secondary index), it returns a flat record with named fields. Blocks do not need fallback logic for missing fields — the system resolves all data before passing it to block evaluation.

Each dataset record is stored with two separate JSON columns:

- **Imported** — populated automatically from an external SQL query on a schedule.
- **Authored** — populated manually by content editors through Chameleon UI.

This separation is a technical optimization: it allows the import pipeline to fully replace imported data without overwriting any authored fields. From the block's perspective, the two columns are invisible — the system merges them into a single flat record before block evaluation.

Field resolution. The system performs a full join across three layers with the precedence: authored → imported → default. If a field exists in any layer, it appears in the merged record. If a field is present in both authored and imported, the authored value wins. The default record provides fallback values for fields missing from both. This means:

- Authors can override any imported value (e.g., fix a misspelled airport name) without waiting for a pipeline fix.
- Authors can add fields that no pipeline produces (FAQs, marketing copy, custom descriptions).
- The default record eliminates the need for per-field fallback logic in blocks.

Soft deletion. When an import removes a record (e.g., an airport is decommissioned), the system soft-deletes the row rather than hard-deleting it. If the record has authored data, the authored fields are preserved. Authors can see soft-deleted records in the UI and restore them manually if needed. This prevents silent loss of editorial work when pipeline data changes.

Dataset configuration:

Field	Description
Name	Unique identifier for the dataset
Primary key	Column used for single-record lookups
Secondary index	Optional column for alternative lookups
Default record	Fallback values for missing fields
QueryBuilder link	Optional — SQL query that populates imported data
Update frequency	Optional — how often the import runs (hourly, daily, weekly)
Localization	Optional list of JSON field selectors specifying which fields are localizable (e.g., <code>["name", "description", "faq.question", "faq.answer"]</code>). Fields not in the list (numbers, URLs, coordinates) are shared across locales and stored once. If empty, the dataset is not localized.

Import workflow:

1. Author writes a SQL query in QueryBuilder and links it to the dataset.
2. On schedule, the system executes the query and stores results in a Hive table.
3. The system compares the new Hive results with the previous import and computes a diff.
4. The diff (added, updated, removed rows) is applied to the dataset's TiDB table, updating only the **imported** JSON column. The **authored** column is untouched. Removed rows are soft-deleted.

Datasets without a pipeline source. Some datasets are purely authored (e.g., promo codes, custom landing page copy). In that case, there is no QueryBuilder link, no Hive table, no import — the **imported** column stays empty. No special case is needed; the model works the same way.

Storage (TiDB):

- **dataset_meta**: dataset_id, name, primary_key, secondary_index, default_record (JSON), querybuilder_link, update_frequency, localized_fields (JSON array of selectors, nullable).
- **dataset_rows**: dataset_id, row_key, locale (nullable), imported (JSON), authored (JSON), deleted_at (nullable).

Chained lookups across datasets (e.g., airport → city → products) are addressed by pre-joining data at pipeline time into denormalized datasets, eliminating the need for

multi-hop lookups at request time. For data already in TiDB, native SQL joins can replace imperative chaining.

Localization

The default approach to localization at Uber is Rosetta: services load a translation snapshot into memory at startup and perform fast in-memory lookups at render time. This works well when the content set is small, static, and has rare updates. Our CMS breaks these assumptions:

1. **Content volume.** Blocks and datasets contain a significant amount of translatable text (descriptions, FAQs, step-by-step instructions) across hundreds of pages. Loading all translations into memory is impractical.
2. **Translation latency.** Authors need near-real-time translations for localized preview. Rosetta's batch snapshot pipeline introduces delays that block the authoring workflow.
3. **Lifecycle management.** When a block or dataset row is removed, the corresponding translations should be removed as well. Rosetta snapshots have no mechanism to garbage-collect obsolete entries.
4. **Author overrides.** Authors need the ability to override machine-generated or imported translations for specific locales — something Rosetta's externally managed snapshots do not support.

Approach. We store source content and translations in TiDB alongside the content they belong to. The system integrates directly with the MT-service (machine translation service managed by the Localization team) to request translations for new or updated content.

How it works:

- When an author publishes a block or edits a dataset record, the system sends the localizable fields (as defined by the dataset's `localized_fields` selectors) to the MT-service.
- The MT-service returns translations, which are stored in TiDB as draft values for author review.
- Authors can review, edit, or override any translation through Chameleon UI.
- At render time, the engine retrieves only the translations needed for the requested page and locale — no full snapshot loaded.
- When a block or dataset row is deleted, its translations are removed (or soft-deleted) automatically.

Change Management

Authors create or modify content through Chameleon Fusion UI. The system combines edits into a **changeSet** containing:

- Layout-override changes (domain, namespace, pattern, block ID, position, weight).
- Block changes (block ID, locale: queries, rules, content mapping).

ChangeSets support draft and published states with full version history.

Preview

Preview rendering compiles Go `.templ` templates to JavaScript using GopherJS, with CSS from EatsWebSEO. When previewing a page, each block is annotated with its provenance ("inherited from /region" or "overridden on this page"), allowing authors to trace unexpected content to its source without mental tree-walking.

Preview flow:

1. Author requests a page for editing in Chameleon Fusion.
2. Chameleon loads the existing version: layout overrides, blocks, data from datasets.
3. Author applies edits (add/remove/modify layout overrides or blocks).
4. Chameleon sends the page request plus the change object to Auteur.
5. Auteur fetches layout overrides, merges with changes, resolves the layout into block IDs, fetches blocks, applies change amendments, queries datasets, maps data to rendering models, and returns the resolved page data.
6. Chameleon renders the resolved page data using compiled templates with provenance annotations.

Preview

Preview allows authors to see how a page will look before publishing. The Auteur Service resolves the layout, merges unpublished changes, queries datasets, and returns resolved page data. The client renders this data using the same Go `.templ` templates and CSS used by EatsWebSEO in production.

The rendering approach for preview is under evaluation in a separate

 [Abstract] Enable Authoring for SEO Pages on ubereats go pages

Options include:

7. Compile `.templ` templates to Go, then compile to JavaScript via GopherJS —

renders in the browser.

8. Server-side rendering via a `render-block` endpoint — Chameleon requests rendered HTML from EatsWebSEO.
9. Compile `.templ` templates to WebAssembly via TinyGo — renders in the browser.

Regardless of rendering approach, the preview annotates each block with its provenance ("inherited from /region" or "overridden on this page"), allowing authors to trace unexpected content to its source.

Preview flow:

1. Author opens a page for editing in Chameleon Fusion.
2. Chameleon loads the current version: layout overrides, blocks, data from datasets.
3. Author applies edits (add/remove/modify layout overrides or blocks).
4. Chameleon sends the page request plus the change object to the Auteur Service.
5. The Auteur Service resolves the layout, merges changes, queries datasets, maps data to rendering models, and returns the resolved page data.
6. The resolved page data is rendered with provenance annotations using one of the approaches above.

APIs and Data

All Auteur Service methods use Protobuf over gRPC. Chameleon Fusion calls them through the Auteur API gateway. Full method signatures are in [apis.md](#).

Page resolution — two methods with different optimization targets:

Method	Called by	Optimized for
<code>ResolvePage(uri, locale)</code>	Sync workflow	Performance — published data only
<code>ResolveChanges(uri, locale, changeset)</code>	Chameleon Fusion	Flexibility — merges unpublished edits

`ResolvePage` returns the fastest possible result: single covering-index query for layout, parallel dataset fetches, no changeset merging. `ResolveChanges` overlays a changeset on published data and annotates each block with its source pattern, whether it was modified, and which fields changed.

Authoring APIs:

Group	Methods	Scope
Layout overrides	List, Get, Create, Update, Delete	Per namespace
Blocks	List, Get, Create, Update, Delete, GetVersions,	Per block ID + locale

Group	Methods	Scope
	GetVersion	
Datasets	List, Get, Create, Update, Delete	Dataset definitions
Dataset rows	Get, List, Update, Delete (soft), Restore, Import, Export	Per dataset + row key + locale
Dataset sync	TriggerSync, GetSyncStatus	Internal, triggered by scheduler
ChangeSets	Create, Get, AddBlockEdit, AddLayoutEdit, RemoveEdit, Publish, Discard, List	Draft → publish flow
Namespaces	List, Create, Delete, ListPermissions, Grant, Revoke	Access control

External API (serving path):

- **GET /content?uri=...&locale=...** — Content Service returns resolved page data for rendering. Populated by the sync workflow via **ResolvePage**; not a direct TiDB query.

Data model (TiDB): Full schema with indexes and queries is in **db_schema.md**.

Table	Key columns
layout_overrides	domain, namespace, pattern, position, weight, block_id
layout_override_versions	override_id, version, block_id, weight
blocks	id, locale, version, query_resource, query_key, content (JSON)
block_versions	id, locale, version, updated_at, updated_by
datasets	id, name, primary_key, secondary_index, default_record, querybuilder_link, update_frequency, localized_fields
dataset_rows	dataset_id, row_key, locale, imported (JSON), authored (JSON), deleted_at
changesets	id, status, created_by, created_at, published_at
changeset_block_edits	changeset_id, block_id, locale, content
changeset_layout_edits	changeset_id, domain, pattern, position, weight, block_id

Integration Design for dependencies

Upstream integrations:

- **Machine Translation Service:** Auteur Service sends new or updated editorial content for translation. Translations are written back to TiDB as drafts for author review.

MT unavailability does not block publishing — it delays translation availability.

Downstream integrations:

- **Content Service / DocStore:** the sync workflow pushes resolved page data to Content Service. Content Service continues to serve EatsWebSEO. No changes to the serving path.
- **EatsWebSEO:** receives resolved page data from Content Service as before. The only change is propagating the changeSet ID for preview rendering.

Failure handling:

- TiDB failure → sync workflow pauses; Content Service serves stale cached content.
- Rolodex failure → affected block queries return null; blocks with rules for missing data degrade gracefully.
- Sync workflow failure → Content Service continues serving the last successfully synced version.

Critical design issues

1. **Partial query failures in parallel execution.** When 10 queries run in parallel and some fail, the engine must handle partial data. Design decision: a failed query returns null for all its fields. Block rules must account for missing data (**when: region == nil → not_found**). The engine logs which queries failed and why.
2. **Layout resolution performance with wildcards.** URL patterns with wildcards (e.g., **/region/*/es**) require pattern matching across potentially many layout-override records. The sync workflow pre-resolves layouts at publish time, so this cost is paid once, not per request. Preview (where pre-resolution is not possible) may require indexing strategies for wildcard patterns.
3. **Data mapping syntax expressiveness vs. complexity.** The syntax must be powerful enough to replace uScript but simple enough that edge cases don't become production incidents. Starting with selectors only and adding features incrementally reduces risk, but existing pages that use complex uScript logic (conditionals, string manipulation, list iteration) need a migration path.
4. **Translation volume.** A dataset with 500 rows × 30 locales produces 15,000 translated cells. Versioning all translations is impractical. Design decision: translations are stored as current values only; recovery relies on MT re-generation or CSV re-import.

Design Best Practices and Considerations

Reliability:

- The serving path (Content Service → EatsWebSEO) has no direct dependency on TiDB or the Auteur Service. All data is pre-resolved and cached. This isolation ensures that authoring-side failures do not affect end-user page rendering.
- The sync workflow is idempotent: re-running it for the same URI produces the same result. Failed syncs can be retried without side effects.

Regional and zonal failover:

- Content Service and DocStore already support regional failover. The sync workflow writes to all regions. TiDB replication handles zonal failover for the authoring path.

Monitoring, Rollout

Monitoring:

- Sync workflow latency and error rate per URI — alerts if sync falls behind or fails repeatedly.
- Query execution latency (p50, p99) per data source — detects degradation in TiDB or upstream services.
- Block evaluation success rate — percentage of blocks that return content vs. null/error.
- Content freshness: time since last successful sync per URI — alerts on stale content.
- Preview latency — ensures authoring experience remains responsive.

KPIs proving value:

- End-to-end page serving latency before and after migration.
- Number of pages managed through the CMS vs. legacy systems.
- Time from content edit to live page (author workflow latency).
- Number of engineering interventions required for content changes.

Migration

Strategy: Incremental migration, one page type at a time. Existing systems continue to serve all current content; the new CMS handles only newly onboarded page types until migration is complete.

Approach:

1. For each page type, translate the existing uScript logic into blocks (queries, rules, content mappings). This is a one-time engineering effort per page type.
2. Run the new system in shadow mode: resolve each URI through both old and new paths, diff the output automatically.
3. When diffs are clean for a sustained period, cut over serving for that page type to the new system.
4. Decommission the old uScript and service integrations for that page type.

Risk mitigation: At every stage, rollback is a feature-flag flip that routes serving back to the old system. No data is deleted from legacy systems until the new system has been stable in production for a defined period.

Content consistency during coexistence: While Auteur and legacy stores (Camopress, Louvre, Rolodex, Content-Entities, Unified-Content) both remain live, author edits in Auteur will not automatically write back to those systems. Dual-writing from Auteur into multiple downstream services is optional, not the default: it is non-trivial to implement and expensive to maintain. Consistency is handled administratively—coordinating which page types and content are migrated, and freezing or routing changes so the same URI is not independently edited in two places. Dual-write is a fallback if coordinated migration is not enough to keep correctness confidence high.

As a guideline, by this point your ERD should be < 30 minutes of reading time (~7 pages). If the level of details becomes too large, consider using the Appendix and/or breaking up your ERD into smaller, more specific ERDs with a narrower but more involved set of stakeholders.

=====

==

Privacy and Security Considerations

=====

==

Data

Processed Personal and Business Data

The proposed service will not serve or store users personal information or business data.

Deletion exception

Not applicable. The service will not store any personal data and does not need any deletion exception.

Data Sharing

Not applicable. The service will not store any personal data and will not share any personal data with 3rd-partes.

Access Control and Rate Limiting

Authentication. Users and services are authenticated based on Charter policies. The Auteur Service runs within the Auteur service boundary and uses AuthFX for service-to-service authentication. Chameleon Fusion authenticates users via standard Uber SSO. Calls to upstream services (Rolodex, MT-service, QueryBuilder) use AuthFX service credentials. No third-party cloud, SaaS, or vendor authentication is required — all integrations are internal Uber services.

Authorization. Every authoring API call includes the caller's identity from the authenticated session. The Auteur Service checks the caller's role for the target namespace before executing the operation. Read-only resolution methods (ResolvePage, ResolveChanges) are internal service-to-service calls authenticated via AuthFX and Charter — they are not exposed to end users.

Role-based access control. Three roles scoped per namespace (domain + first URL segment):

Role	Permissions
Viewer	Read-only access to layout overrides, blocks, datasets, and changesets within the namespace. All authenticated users have Viewer access.
Editor	Create, update, and delete layout overrides and blocks. Edit

Role	Permissions
	dataset rows (authored data). Create and publish changesets. Cannot manage namespaces or permissions.
Admin	All Editor permissions, plus: create and delete namespaces, grant and revoke permissions for other users.

Role assignments are stored in TiDB and managed through the Namespaces & Permissions API. The service validates the caller's role on every mutating request.

Service and storage authentication. All service-to-service calls are authenticated and authorized via AuthFX and Charter. TiDB access is restricted to the Auteur service identity — no direct database access by employees or other services. Employee access to authoring APIs goes through Chameleon Fusion, which authenticates via SSO and passes the user identity to the Auteur Service for role-based authorization. Administrative access to TiDB for debugging or operational tasks follows standard Uber data access policies and requires explicit approval.

Rate limiting.

Endpoint type	Approach
ResolvePage (sync workflow)	Muttley Rate Limiter, per calling service identity. Burst capacity sized for full-site re-sync.
ResolveChanges (Chameleon preview)	Muttley Rate Limiter, per service. Expected: tens of requests per author per session.
Authoring CRUD APIs (Chameleon)	Muttley Rate Limiter, per service Expected: single-digit RPS per author.

Handling User/External Input

The service will accept input from internal authors only, and the input will be strictly validated versus schema based data models.

LLMs

The service will not utilize any LLM or MCP.

Logging and Monitoring

Access logging. The Auteur Service leverages AuthFX out-of-the-box access logs, which capture {Actor, Resource, Action} for every authenticated request. This provides a complete audit trail of who accessed or modified what and when. Specifically:

- Every mutating operation (create, update, delete on layout overrides, blocks, dataset rows, and changesets) is logged with the authenticated user identity, the target resource ID, the action performed, and a timestamp.
- Changeset publish events are logged with the full list of affected resource IDs, enabling forensic reconstruction of what changed in a given publish action.
- Permission changes (GrantPermission, RevokePermission) are logged as administrative actions with both the granting user and the target user.

Data sensitivity. The Auteur Service does not store or process sensitive personal data (government IDs, user profile data, payment information, driver/rider PII). All content managed by the CMS is editorial and operational in nature — page layouts, marketing copy, airport descriptions, city metadata, ride product names. Dataset imports from Hive contain aggregated operational data (ratings, trip counts, pricing), not individual user records. As a result, the risk profile for excessive-read monitoring is low, but we still monitor for anomalous access patterns.

Monitoring and alerting. The service will be onboarded onto Platform Alerts.

=====

==

APPENDIX

Anything else that people might want to know. This section is not subject to any page limits, but this is “optional” reading and people should be able to understand the important points without it. It is encouraged to use hyperlinks in the main document - both to the relevant appendices and to other documents.

=====

==

Assumptions

- **Fragmentation, not latency, is the bigger problem.** If latency were the only issue, [the LLM-optimization alternative](#) would win on cost and time-to-value. The whole design pivots on this being wrong.
- **Content Service (DocStore) is a viable serving cache.** Pages are pre-resolved on publish/sync and served from cache; TiDB is on the authoring path only.
- **TiDB fits the shape.** Covering-index layout resolution, per-locale rows, and sync-generation semantics benefit more from TiDB than from DocStore existing storage.
- **expr-lang is expressive enough for block mapping.** Blocks are data mappers, not programs — no loops, no side effects. If real templates require more, we revisit the language, not the block model.
- **One-level query dependency depth is acceptable.** Authors can denormalize at import time when they need two hops. Enforced at publish time with a hard error.
- **Rendering stays in EatsWebSEO with typed Go models.** Auteur returns typed protobuf, never HTML.
- **Localization stays with MT-service.** Auteur stores and serves translations but doesn't generate them.
- **URL preprocessing (segment normalization, MD5 for non-ASCII / >16-char segments) lives inside Auteur.** Callers pass raw URLs.

Alternatives

- **Keep the current stack (uScript + Louvre + Rolodex + Typewriter).** Rejected — leaves the fragmented authoring surface untouched. Content team still needs three engineers and four tools to launch a new city.
- **LLM-assisted uScript optimization.** Ships latency wins in 2–4 weeks with zero migration risk, but leaves fragmentation, cross-block dedup, layout overrides, changesets, and preview provenance unsolved. **Recommended as a hybrid bridge,** not a replacement.

FAQ

1. **Why aren't we doing this for other pages too?** UberEats SEO is the beachhead — most pressing latency problem, clearest content-team ownership. Namespaces and

domains are first-class, so Rides adoption is a config change after V1 lands.

2. **Why not use Camopress itself as the CMS?** Since we are going to change all the components (primary storage, layout resolution, block expression language, dataset management) it is easier to build a new service than gradually modify existing code.
3. **Are we replacing Typewriter?** No. Typewriter continues to feed data via QueryBuilder → dataset imports. Auteur overlays authored edits on top of imported rows.
4. **Why TiDB and not the DocStore?** DocStore is tuned for template blobs, not covering-index layout resolution or per-row locale variants. Forking that schema would hurt existing workloads.
5. **Why expr-lang and not Starlark/Lua/JS?** Blocks are data mappers, not programs. Expression-only, no loops, no side effects, bytecode-compiled, safe to run in-process on every request.
6. **What happens to existing pages during migration?** They keep serving from Camopress uScript until the block equivalents pass shadow-diff. Migration is per page-type, reversible via Flipr.
7. **Who owns Auteur on-call?** SEO Platform team — the same team that owns Camopress today. No new on-call rotation.
8. **How do we prevent SEO regressions during cutover?** Shadow-mode diff between old and new pipelines for N days per URL pattern. Cutover only after diff is clean; rollback is a Flipr flip.

Points of Contention

- **Is fragmentation actually the bigger problem, or is latency?** This is the load-bearing claim of the whole ERD. **Resolution:** sync with content team leadership on launches blocked by tooling this year vs. customers lost to p99. If latency dominates, downscope to the LLM optimizer.
- **Depth-1 dependency limit is too strict.** Some pages walk two hops today (airport → city → country → regulation). We push denormalization to the import pipeline; some engineers argue that shifts complexity, not removes it. **Resolution:** audit the top 20 page templates and count >1-hop cases. If <5%, ship the constraint; if >20%, raise the limit or add compiler-inserted joins.
- **Content Service (DocStore) as serving cache.** Some argue it isn't tuned for block-shaped payloads. **Resolution:** prototype with DocStore; introduce a purpose-built cache only if p99 shows tail problems DocStore can't fix.
- **Chameleon Fusion as authoring UI.** Some argue it's too coupled to legacy Camopress patterns. **Resolution:** API-first design keeps Auteur UI-agnostic; swap the

UI if Fusion underdelivers.

If any of these can't be closed in review, escalate the specific question to paired managers and senior ICs across SEO Platform, Content, and DB Infrastructure. Don't stall the whole ERD — carry unresolved points forward as tracked risks, not blockers.

Cost considerations

Direct infrastructure

- **TiDB**: ~100–500 GB V1; ~100–500 read RPS + 5–10 write RPS during authoring hours. Same shape as existing SEO Platform TiDB tenants.
- **Compute**: 3–5 Go service instances V1, scaling to 10–15 with adoption.
- **Cold storage**: **block_versions** + **layout_override_versions** grow ~50 MB/month at V1 scale.

Downstream

- **Content Service (DocStore)**: 30–50 sync-write RPS; ~10 GB/quarter growth.
- **Rolodex**: unchanged during migration; drops post-migration as datasets move to TiDB imports.
- **MT-service**: 1–5 RPS bursty, absorbed by existing capacity.
- **QueryBuilder / Hive**: 10–50 batch queries/hour, absorbed by existing capacity.
- **AuthFX / Charter / Cadence**: per-request identity checks and sync workflow starts, absorbed by existing capacity.

RU estimate (needs Resource Insights validation before Ph2 budget review)

- **Current RU/yr**: SEO CMS-shaped work is spread across Camopress uScript, Louvre hydration, Typewriter/Rolodex reads, and Unified-Content storage. Rough order: ~5–10M RU/yr attributable to SEO Platform across these systems.
- **Estimated new RU/yr**: ~7–12M RU/yr during migration (both stacks running); settling to ~4–7M RU/yr post-migration as Camopress uScript execution retires.
- **Estimated increase RU/yr (peak, during migration)**: ~+2–4M RU/yr for 6–12 months, then net decrease.

The largest cost risk is not the steady state but the length of the transient double-cost window. If migration slips past 18 months, that window becomes the dominant line item —

watch migration velocity more than steady-state RU.

Other considerations

OTHER CONSIDERATIONS

Service Dependencies

Dependency	On request path?	Failure impact on end users	Estimated additional RPS
TiDB	Authoring + sync path only	None — Content Service serves cached data	100–500 read, 5–10 write
Content Service	Sync write path only	None — operates independently	30–50 write
Rolodex	Sync path (ResolvePage)	None — cached data served until next successful sync	50–200 read
MT-service	Async only	None — source locale served as fallback	1–5
QueryBuilder / Hive	Batch only	None — last import data retained	N/A (batch)

Key point: No dependency is on the end-user serving path. All dependencies affect either the authoring experience (TiDB) or content freshness (Rolodex, QueryBuilder, MT-service). End-user page rendering is served entirely from Content Service cache, which operates independently of all Auteur Service dependencies.

Impact of migration

This proposal does not require migration. We will preserve all existing capabilities of Camopress, Typewriter and other services as is.

The new SEO pages and datasets will be created by authors. They may replace existing content in future.

Metrics, Monitoring, and Pager-Duty Rotation

All metrics are emitted via the M3 client from the Camopress service.

Metric	Tags	Purpose
<code>auteur.resolve_page.latency</code>	<code>domain, namespace, status</code>	ResolvePage p50/p99 latency
<code>auteur.resolve_page.count</code>	<code>domain, namespace, status</code>	ResolvePage request rate and success/error split

Monitoring

Dashboards (Grafana):

Dashboard	Contents	Audience
Auteur Service Health	ResolvePage/ResolveChanges latency and error rates, TiDB latency, sync freshness	Auteur team (primary), Camopress on-call
Sync Workflow	Per-namespace sync latency, success rate, content freshness, queue depth	Auteur team
Dataset Imports	Per-dataset sync status, rows added/updated/removed, import failures	Auteur team, content operations
Authoring Activity	Changeset publishes, CRUD operations, active authors per namespace	Auteur team, content team leads

Alerting

Responsible team: Camopress on-call rotation (part of the Web Platform team). The Auteur Service is deployed within Camopress and follows its existing on-call structure.

Planned alerts (first 3 months):

Alert	Condition	Severity
ResolvePage error rate high	Error rate > 1% over 5 minutes	P2
ResolvePage latency degraded	p99 > 500ms over 5 minutes	P3
Sync workflow stalled	No successful sync for any namespace in 15 minutes	P2
TiDB unreachable	Health check fails for 2 consecutive checks	P1
Dataset import failure	Any dataset sync fails 3 consecutive times	P3

Alert	Condition	Severity
Changeset publish failure	Any publish operation fails	P3
Content freshness exceeded	Any namespace's content older than 30 minutes	P2

Approximately 7–10 alerts in the first 3 months. This count does not scale with trips or cities — alerts are based on system health thresholds, not traffic volume. As new page types are migrated, we may add per-page-type freshness alerts, but the total will remain under 20.

Service Level Objectives

Latency:

- ResolvePage: p50 < 50ms, p99 < 200ms. This is the sync workflow path, not the end-user serving path. End-user latency is governed by Content Service SLOs.
- ResolveChanges (preview): p50 < 200ms, p99 < 1s. Preview is an authoring-time operation with higher latency tolerance.
- CRUD APIs: p50 < 50ms, p99 < 200ms. Standard for internal authoring operations.

Availability:

- Auteur Service: 99.9% availability (3-nines). Auteur downtime does not affect end-user page serving — Content Service continues to serve cached data. Auteur downtime affects authoring (authors cannot edit or preview) and content freshness (sync pauses).
- Content Service (serving path): governed by existing Content Service SLOs (99.99%). The Auteur Service does not change Content Service availability characteristics.

Throughput:

- ResolvePage: designed for up to 100 RPS sustained (10K URIs synced every ~100 seconds). Burst capacity to 500 RPS for full-site re-sync.
- ResolveChanges: designed for up to 20 RPS (20 concurrent authors previewing). Not expected to exceed 10 RPS in practice.
- CRUD APIs: designed for up to 50 RPS. Not expected to exceed 5 RPS in practice.

Fraud

Does this project impact the fraud rate at Uber? No. The Auteur Service manages

editorial content (page layouts, marketing copy, airport descriptions, city metadata). It does not process transactions, payments, user accounts, ride requests, or any financial operations.

Users and fraud scenarios: The users are internal Uber content editors and engineers, authenticated via SSO. Potential misuse scenarios:

- An editor with compromised credentials could publish misleading content (wrong pricing, fake promotions). This is mitigated by changeset audit logging, publish notifications, and the ability to rollback via feature flag.
- An editor could attempt to inject malicious content (XSS via authored text fields). This is mitigated by EatsWebSEO's rendering layer, which escapes all content before rendering to HTML. The Auteur Service stores data, not HTML — rendering-time sanitization is the responsibility of EatsWebSEO.

Audit logging: Yes. All mutating operations are logged via AuthFX access logs ({Actor, Resource, Action}). Changeset publish events include the full list of affected resources. See the Security Logging and Monitoring section for details.

Documentation

Document	Location	Audience	Maintenance
ERD (this document)	Google Docs / Confluence	Engineering, architecture reviewers	Updated during design review; archived after approval
API reference	Auto-generated from Protobuf definitions, published to internal docs portal	Engineers integrating with the Auteur Service	Updated automatically on each deploy
Block authoring guide	Confluence / Chameleon help section	Content editors, content operations	Maintained by the Auteur team; updated when block structure or expression language changes
Dataset management guide	Confluence / Chameleon help section	Content editors, data analysts	Maintained by the Auteur team; updated when dataset configuration or import workflow changes
Runbook	Confluence, linked from on-call rotation	Camopress on-call engineers	Maintained by the Auteur team; updated after each incident
TiDB schema reference	db_schema.md in the project	Engineers	Updated with each schema

Document	Location	Audience	Maintenance
	repository		migration
Grafana dashboards	Grafana (linked from runbook)	On-call, content operations	Maintained alongside alerting configuration

Support

The proposed system will not impact Uber support since it is intended for internal authors only.
Existing #dotcom-support Slack channel is enough for communications and authors support.

Open Source

Not applicable. The proposal is not intended to be open source or hosted on a public Github repository.

Patents

Not applicable. The proposal does not include any new and useful patentable inventions.

Data Persistence and Retention

Data persisted by the Auteur Service:

Data	Storage	Estimated size	Growth rate
Layout overrides + versions	TiDB	Small — thousands of records	Low — changes only on publish
Blocks + versions	TiDB	Moderate — thousands of blocks × locales × versions	Low — grows with publishes
Dataset definitions	TiDB	Small — tens to low hundreds of datasets	Negligible
Dataset rows (imported + authored)	TiDB	Moderate to large — up to 10 ⁶ rows total across all datasets	Depends on import frequency; daily imports for active datasets
Changesets + edits	TiDB	Moderate — one changeset per publish	Proportional to publish

Data	Storage	Estimated size	Growth rate
		action	frequency
Hive staging tables	HDFS/Hive	Mirrors dataset import size — up to 10 ⁶ rows per dataset	Replaced on each sync cycle

Retention policies:

Data	Retention	Rationale
Published blocks and layout overrides (current version)	Indefinite — retained as long as the page is active	Required for serving
Block versions and layout override versions	Last 50 versions per (block_id, locale) pair	Sufficient for audit trail and rollback; older versions are rarely needed
Dataset rows (current)	Indefinite — retained as long as the dataset is active	Required for serving
Soft-deleted dataset rows	90 days after deletion	Gives authors time to restore accidentally removed records; after 90 days, hard-deleted
Changesets (published)	1 year	Audit trail for compliance; after 1 year, changeset metadata retained but edit payloads deleted
Changesets (draft/discarded)	30 days	Stale drafts are cleaned up automatically
Hive staging tables	Retained for 2 most recent sync generations per dataset	Previous generation kept for diff computation; older generations dropped

Enforcement: Retention policies are enforced by a scheduled cleanup job running within the Auteur Service. The job runs daily, identifies records past their retention window, and deletes them in batches to avoid TiDB hotspots. Hive table retention is managed by the dataset sync workflow — old generations are dropped after a successful sync.

Data Quality

Dataset import pipeline:

1. Author configures a dataset with a QueryBuilder link (SQL query against Hive).
2. On schedule, the Auteur Service submits the SQL query to QueryBuilder.
3. QueryBuilder executes the query and writes results to a Hive staging table scoped to the dataset.

4. The Auteur Service reads the Hive staging table and computes a diff against the current TiDB data (added, updated, removed rows).
5. The diff is applied to TiDB: new rows inserted, changed rows updated (imported column only), removed rows soft-deleted. The authored column is never touched by the import.

Impact on existing pipelines and reporting: None. The Auteur Service reads from existing Hive tables via QueryBuilder queries authored by content editors. It does not modify any upstream data sources, does not produce events consumed by other pipelines, and does not alter any existing ETL flows. The datasets stored in TiDB are internal to the CMS and are not consumed by analytics or reporting pipelines outside of page rendering.

Data quality checks during import:

- Row count comparison: if the new import has significantly fewer rows than the previous generation (>50% drop), the sync is paused and an alert is raised. This prevents a broken query from wiping a dataset.
- Schema validation: imported JSON is validated against the dataset's expected columns. Rows with missing required columns are logged and skipped.
- Duplicate key detection: if the import contains duplicate primary keys, only the first row is retained; duplicates are logged.
- Sync status is recorded per dataset (rows added, updated, removed, skipped) and exposed via [GetSyncStatus](#) for monitoring.

Testability

Unit tests:

Component	What is tested
Layout resolution	Pattern matching with wildcards, weight-based override selection, correct block ordering by position
Field merging	Authored → imported → default precedence, missing fields, null handling, full-join behavior
Block rules evaluation	Redirect, not_found, null conditions with various data states, rule ordering
Content mapping	Field selector extraction, expression evaluation (if applicable), each/as iteration
Dataset sync diff	Add, update, soft-delete logic, row count threshold check, duplicate key handling
Changeset merge	Overlay of draft edits on published data, conflict detection

Integration tests:

Scenario	What is tested
ResolvePage end-to-end	URI → layout resolution → dataset queries → field merging → rendering model output. Validates against known expected output for representative pages (airport, city, region).
ResolveChanges with changeset	Same as ResolvePage but with a changeset applied. Validates that origin annotations are correct and modified fields are flagged.
Dataset import cycle	QueryBuilder SQL → Hive staging → diff → TiDB upsert. Validates row counts, soft-deletion, authored column preservation.
Changeset publish	Draft → publish → verify blocks and layout overrides tables are updated atomically. Verify version history is created.
Permission enforcement	Editor cannot modify resources outside their namespace. Viewer cannot perform mutating operations. Admin can grant/revoke roles.

Performance tests:

- Layout resolution query latency under load (target: sub-10ms p99 for the covering-index query).
- Parallel dataset fetch latency with varying numbers of concurrent queries (5, 10, 20).
- Changeset merge overhead compared to ResolvePage baseline.

Deployability

Deployment target. The Auteur Service deploys as part of the existing Camopress service using Camopress's standard deployment pipeline (uDeploy). No new service infrastructure is required — the Auteur code, APIs, and handlers are added to Camopress as a separate module with its own TiDB database.

Database migrations. TiDB schema changes are managed through versioned migration scripts applied via the standard Uber database migration tooling. Migrations are backward-compatible: new columns are added as nullable, new tables are additive, and no existing columns are dropped until the previous version is fully decommissioned.

Maps Data - Data Review Documentation (DRD)

Not applicable. The proposal does not affect Maps data.

Earner Data - Data Review Documentation (DRD)

Not applicable. The proposal does not affect Earner data.

Delivery Data - Data Review Documentation (DRD)

Not applicable. The proposal does not affect Delivery data.

Fares Data - Data Review Documentation (DRD)

Not applicable. The proposal does not affect Fares data.

Generative AI

Not applicable. The proposal does not include any Generative AI components.

Financial (SOX)

Not applicable. The proposal will not affect Driver Payments, Fare Calculation, or Money Movement.

Earner Access Config - Proposed configuration changes

Not applicable. The proposal will not affect Earner Access Configuration.

Database Schema

Database Schema

TiDB is the primary database. Schema design follows TiDB distribution principles: random UUID v4 primary keys (or deterministic UUID v5 from source columns) to avoid write hotspots; covering indexes for layout resolution; per-locale rows for content and dataset records; and no reliance on foreign key enforcement (handled in application code).

Changesets are the unit of versioning — `block_versions`, `layout_overrides_versions`, and `dataset_meta_versions` reference `changeset_id` and carry the historical row content.

Table-level charset and collation are omitted; every table inherits from the database defaults.

Blocks

`blocks` stores the current published version of a block for fast reads. Draft and historical rows live in `block_versions`, keyed by `changeset_id`. On publish, the changeset's rows are copied into `blocks` and the `changeset_id` on the canonical row identifies the publish event.

Expected 10^5 records per locale, $\sim 10^6$ records total.

SQL

```
CREATE TABLE blocks (  
  `id`          BINARY(16) NOT NULL, -- uuid v4 random  
  `locale`      VARCHAR(8) NOT NULL, -- BCP47, e.g. "en-US"  
  `changeset_id` BINARY(16) NOT NULL,  
  `model`       VARCHAR(32) NOT NULL,  
  `content`     BLOB       NOT NULL, -- protobuf-encoded block content  
  PRIMARY KEY (`id`, `locale`) CLUSTERED  
);
```

SQL

```
/*  
 * block_versions stores the changes to a block per changeset.  
 * Expected ~10 records per changeset, 10^5 records total.  
 */
```

```

/*
 * Index idx_block_versions_changeset_id: fetch block versions by changeset_id.
 * Index idx_block_versions_block_id_locale: fetch versions of a block.
 */
CREATE TABLE block_versions (
  `id`          BINARY(16)  NOT NULL, -- uuid v4 random
  `changeset_id` BINARY(16)  NOT NULL,
  `block_id`     BINARY(16)  NOT NULL,
  `locale`       VARCHAR(8)  NOT NULL,
  `updated_at`   TIMESTAMP(6) NOT NULL,
  `model`        VARCHAR(32) NOT NULL,
  `content`      BLOB        NOT NULL,
  PRIMARY KEY (`id`) CLUSTERED,
  INDEX idx_block_versions_changeset_id (`changeset_id`),
  INDEX idx_block_versions_block_id_locale (`block_id`, `locale`,
  `updated_at` DESC)
);

```

Layout Overrides

`layout_overrides` maps normalized URL patterns to block IDs per page position. Weight determines specificity — when multiple overrides match the same position, the highest weight wins. Locale is not stored here; it comes from the request and is applied when joining to `blocks`.

URL preprocessing normalizes the path into 4 slots. A pattern key like `"san-francisco/airport//"` matches any value in the third and fourth slots. See the layout resolution query below for how permutations are generated at read time.

Expected $\sim 10^5$ records total.

SQL

```

/*
 * id: UUID v5 deterministic from (domain_raw, prefix_raw, pattern_raw,
 * position)
 *
 * domain, prefix, pattern: SHA1 digests of canonical lookup keys
 * (computed in application code via IndexKey). Not human-readable.
 * - domain:  SHA1(domain_raw) - always present, fixed 20 bytes → BINARY(20)
 * - prefix:  SHA1(prefix_raw); x'' for domain-wide defaults → VARBINARY(20)
 * - pattern: SHA1(canonical pattern key); x'' for prefix-wide defaults
 *
 */

```

```

* VARBINARY on prefix/pattern allows storing x'' as the wildcard value.
* SHA1 digests keep the covering index compact and support ad-hoc queries via
* SHA1() SQL functions.
*
* position: page position, e.g. 1 for header, 9 for footer
* weight:    specificity - highest wins per position
* block_id: block ID without locale
*
* Index idx_layout_overrides_covering: layout resolution without table
lookups.
* Index idx_layout_overrides_block_id: reverse lookup by block_id.
*/
CREATE TABLE layout_overrides (
  `id`          BINARY(16)    NOT NULL,
  `domain`      BINARY(20)    NOT NULL,
  `prefix`      VARBINARY(20) NOT NULL,
  `pattern`     VARBINARY(20) NOT NULL,
  `position`    INT UNSIGNED  NOT NULL,
  `weight`      INT UNSIGNED  NOT NULL,
  `block_id`    BINARY(16)    NOT NULL,
  `updated_at`  TIMESTAMP(6)  NOT NULL,
  `domain_raw`  VARCHAR(64)   NOT NULL,
  `prefix_raw`  VARCHAR(32)   NOT NULL,
  `pattern_raw` VARCHAR(1024) NOT NULL,
  PRIMARY KEY (`id`) CLUSTERED,
  INDEX idx_layout_overrides_covering (`domain`, `prefix`, `pattern`,
`position`, `weight`, `block_id`),
  INDEX idx_layout_overrides_block_id (`block_id`)
);

```

SQL

```

/*
* layout_overrides_versions stores the changes to a layout override per
changeset.
* Expected ~10 records per changeset, 10^5 records total.
*
* Index idx_layout_overrides_changes_changeset_id: reverse lookup by
changeset_id.
* Index idx_layout_overrides_changes_layout_override_id: history for a
specific override.
*/
CREATE TABLE layout_overrides_versions (
  `id`          BINARY(16)    NOT NULL, -- uuid v4 random
  `changeset_id` BINARY(16)    NOT NULL,
  `layout_override_id` BINARY(16) NOT NULL,

```

```

`domain`          BINARY(20)    NOT NULL,
`prefix`          VARBINARY(20) NOT NULL,
`pattern`         VARBINARY(20) NOT NULL,
`position`        INT UNSIGNED NOT NULL,
`weight`          INT UNSIGNED NOT NULL,
`block_id`        BINARY(16)    NOT NULL,
`updated_at`      TIMESTAMP(6)  NOT NULL,
`domain_raw`      VARCHAR(64)   NOT NULL,
`prefix_raw`      VARCHAR(32)   NOT NULL,
`pattern_raw`     VARCHAR(1024) NOT NULL,
PRIMARY KEY (`id`) CLUSTERED,
INDEX idx_layout_overrides_changes_changeset_id (`changeset_id`),
INDEX idx_layout_overrides_changes_layout_override_id
(`layout_override_id`, `updated_at` DESC)
);

```

Layout Resolution Query

Given a URL like `ubereats.com/region/san-francisco/airport/sfo`, the engine generates all pattern permutations by replacing each segment with an empty slot. For a 4-segment path `/1/2/3/4`, this produces 16 patterns:

None

```

'////', '/1///', '/2///', '///3', '///4', '/1/2//', '/1//3', '/1///4',
'//2/3/', '//2//4', '///3/4', '/1/2/3/', '/1/2//4', '/1//3/4', '//2/3/4',
'/1/2/3/4'

```

The query uses a window function to pick the highest-weight override per position, then joins to `blocks` for content:

SQL

```

SELECT b.*
FROM (
  SELECT position, block_id
  FROM (
    SELECT
      position,
      block_id,
      ROW_NUMBER() OVER (
        PARTITION BY position

```

```

        ORDER BY weight DESC, block_id ASC
    ) AS rn
FROM layout_overrides
WHERE domain = :domain
    AND prefix = :prefix
    AND pattern IN (:patterns)
) ranked
WHERE rn = 1
) layout
JOIN blocks b
    ON b.id = layout.block_id
    AND b.locale = :locale
ORDER BY layout.position;

```

This executes as a single query using the covering index — no application-level tree walking. Locale fallback (e.g. `es-MX` → `es-ES` → `en-US`) is handled by expanding `b.locale = :locale` into `b.locale IN (:chain)` plus a per-block ranked pick.

Dataset Meta

`dataset_meta` is the dataset definition. It is small ($\sim 10^2$ records) and rarely changed, so it participates in the changeset flow via `dataset_meta_versions`.

SQL

```

/*
 * dataset_meta stores the metadata for a dataset.
 * Expected 10^2 records total.
 *
 * id: UUID v5 deterministic from (name)
 * content_mode is an ENUM enforced in application code ("overwrite" |
"append").
 *
 * Index idx_dataset_meta_id_updated_at: history / last-updated queries per
dataset.
 */
CREATE TABLE dataset_meta (
    `id`          BINARY(16)    NOT NULL, -- uuid v5
    `updated_at`  TIMESTAMP(6)  NOT NULL,
    `updated_by`  BINARY(16)    NOT NULL,
    `name`        VARCHAR(64)   NOT NULL,
    `description` VARCHAR(1024)  NOT NULL,
    `content_mode` VARCHAR(16)   NOT NULL, -- "overwrite" or "append"

```

```

    `default_record` JSON, -- fallback values; exclude from CDC consistency
    checker
    PRIMARY KEY (`id`) CLUSTERED,
    INDEX idx_dataset_meta_id_updated_at (`id`, `updated_at` DESC)
);

```

SQL

```

/*
 * dataset_meta_versions stores the changes to a dataset meta per changeset.
 * Expected ~10 records per changeset, 10^5 records total.
 *
 * Index idx_dataset_meta_changes_changeset_id: fetch dataset meta versions by
 * changeset.
 * Index idx_dataset_meta_changes_dataset_id: history for a specific dataset.
 */
CREATE TABLE dataset_meta_versions (
  `id` BINARY(16) NOT NULL, -- uuid v4 random
  `changeset_id` BINARY(16) NOT NULL,
  `dataset_id` BINARY(16) NOT NULL,
  `updated_at` TIMESTAMP(6) NOT NULL,
  `name` VARCHAR(32) NOT NULL,
  `description` VARCHAR(1024) NOT NULL,
  `content_mode` VARCHAR(16) NOT NULL,
  `default_record` JSON, -- exclude from CDC consistency checker
  PRIMARY KEY (`id`) CLUSTERED,
  INDEX idx_dataset_meta_changes_changeset_id (`changeset_id`),
  INDEX idx_dataset_meta_changes_dataset_id (`dataset_id`, `updated_at` DESC)
);

```

Dataset Records

`dataset_records` holds all records for all datasets in a single table. Each record has two JSON columns: `imported` (populated by the pipeline) and `authored` (populated by editors). The application merges them at read time with precedence: `authored` → `imported` → `default`.

Records are **not versioned**: automatic pipeline updates plus record volume (10^8 expected) make per-record change history impractical. The Hive partition history serves as the audit trail for `imported`; author edits to `authored` are captured by

application-level activity logs, not by TiDB row versioning.

The primary key is a UUID v5 derived from (`dataset_id`, `unique_key`, `locale`), which makes the row addressable without a composite lookup while keeping writes randomly distributed across TiDB shards.

SQL

```
/*
 * dataset_records stores the records for all datasets in a single table.
 * Expected 10^8 records total. Rows are refreshed daily/weekly/monthly.
 *
 * Optimized for two query shapes:
 * 1. Fetch a single record by (dataset_id, unique_key, locale).
 * 2. Fetch multiple records (up to 1000) by (dataset_id, group_key, locale).
 *
 * id: UUID v5 deterministic from (dataset_id, unique_key, locale).
 *
 * Index idx_dataset_records_dataset_id_group_key_locale: batch fetch by group.
 * Index idx_dataset_records_dataset_id_updated_at: recent-changes queries per
dataset.
 */
CREATE TABLE dataset_records (
  `id`      BINARY(16) NOT NULL, -- uuid v5
  `updated_at` TIMESTAMP(6) NOT NULL,
  `dataset_id` BINARY(16) NOT NULL,
  `unique_key` BINARY(20) NOT NULL,
  `group_key` BINARY(20) NOT NULL,
  `locale`   VARCHAR(8) NOT NULL,
  `imported` JSON NOT NULL, -- exclude from CDC consistency checker
  `authored` JSON NOT NULL, -- exclude from CDC consistency checker
  PRIMARY KEY (`id`) CLUSTERED,
  INDEX idx_dataset_records_dataset_id_group_key_locale (`dataset_id`,
`group_key`, `locale`),
  INDEX idx_dataset_records_dataset_id_updated_at (`dataset_id`, `updated_at`
DESC)
);
```

Point lookup by unique key

The application derives `id = UUID_v5(dataset_id, unique_key, locale)` and reads directly by primary key:

SQL

```
SELECT r.imported, r.authored, m.default_record
FROM dataset_records r
JOIN dataset_meta m ON m.id = r.dataset_id
```

```
WHERE r.id = :id;
```

Locale fallback (e.g. **es-MX** → **es-ES** → **en-US**) is expressed by computing the fallback chain of **id** values in application code and issuing a single **IN (...)** read.

Batch lookup by group key (~1000 rows)

SQL

```
SELECT r.unique_key, r.locale, r.imported, r.authored
FROM dataset_records r
WHERE r.dataset_id = :dataset_id
      AND r.group_key = :group_key
      AND r.locale IN (:locale_chain);
```

The application merges the three layers (authored → imported → default) after retrieval, then applies the fallback chain to pick a winning row per **unique_key**.

Changesets

changesets groups draft edits for atomic publish. A changeset is scoped to (**domain**, **prefix**) — the authoring namespace — so unrelated authoring surfaces do not collide. On publish, the associated **block_versions**, **layout_overrides_versions**, and **dataset_meta_versions** rows are promoted into their canonical tables in one transaction and the **published** flag flips.

Expected ~10⁵ records total.

SQL

```
/*
 * Index idx_changesets_updated_at: reverse chronological listing.
 * Index idx_changesets_updated_by: filter by author.
 */
CREATE TABLE changesets (
  `id`          BINARY(16)    NOT NULL, -- uuid v4 random
  `updated_at`  TIMESTAMP(6)  NOT NULL,
  `updated_by`  BINARY(16)    NOT NULL,
  `domain`      BINARY(20)    NOT NULL,
  `prefix`      VARBINARY(20) NOT NULL,
```

```

`domain_raw` VARCHAR(64) NOT NULL,
`prefix_raw` VARCHAR(32) NOT NULL,
`published` BOOLEAN NOT NULL,
PRIMARY KEY (`id`) CLUSTERED,
INDEX idx_changesets_updated_at (`updated_at` DESC),
INDEX idx_changesets_updated_by (`updated_by`)
);

```

Block Queries (derived dependency index)

block_queries is a derived index of the data queries a published block issues. Block **content** is stored as a Protobuf blob and may issue 0–10 queries against datasets (and optionally external service resources). On publish, application code decodes the block and upserts one row per query. Draft blocks are not indexed.

The primary use is reverse lookup — given a dataset (and optionally an external resource), find the blocks that read from it, and then via **layout_overrides** find the pages those blocks appear on. This answers "which pages are affected when a dataset changes?" without decoding every block at query time.

SQL

```

/*
 * dataset_id: target dataset for internal queries; set for all rows.
 * resource: SHA1(resource_raw); x'' when the query is internal to
dataset_id.
 *
 * Non-empty when the block also calls an external service API.
 * resource_raw: human-readable resource identifier (e.g. external API name).
 *
 * Index idx_block_queries_block_id_locale: all queries for a block.
 * Index idx_block_queries_dataset_id_resource: reverse lookup by dataset (and
resource).
 */
CREATE TABLE block_queries (
  `id` BINARY(16) NOT NULL, -- uuid v4 random
  `block_id` BINARY(16) NOT NULL,
  `locale` VARCHAR(8) NOT NULL,
  `dataset_id` BINARY(16) NOT NULL,
  `resource` VARBINARY(20) NOT NULL,
  `resource_raw` VARCHAR(256) NOT NULL,
  PRIMARY KEY (`id`) CLUSTERED,
  INDEX idx_block_queries_block_id_locale (`block_id`, `locale`),

```

```
INDEX idx_block_queries_dataset_id_resource (`dataset_id`, `resource`)
);
```

Schema Notes

- **No foreign keys at runtime.** TiDB parses FK syntax but does not enforce it. Referential integrity is handled in application code.
- **UUID v4 for random IDs, UUID v5 for deterministic IDs.** Random v4 distributes writes across TiDB shards, avoiding the write-hotspot problem that time-ordered v7 would cause. Deterministic v5 (from source columns) is used where an identity must be recomputable — `layout_overrides.id` from `(domain_raw, prefix_raw, pattern_raw, position)`, `dataset_meta.id` from `(name)`, `dataset_records.id` from `(dataset_id, unique_key, locale)`.
- **Changesets are the unit of versioning.** `block_versions`, `layout_overrides_versions`, and `dataset_meta_versions` reference `changeset_id`; entity rows carry `changeset_id` (blocks) so the publisher that produced them is identifiable. There is no per-entity monotonic version column.
- **Dataset records are not versioned.** Volume (10^8) and pipeline-driven update frequency make per-row version history impractical. History for `imported` lives in Auteur's Hive partitions; history for `authored` lives in application-level activity logs.
- **Per-locale rows.** Both `blocks` and `dataset_records` store one row per locale. Locale fallback is a ranked `IN (...)` read at query time, not a JSON blob lookup.
- **Covering index on layout_overrides.** `idx_layout_overrides_covering` contains all columns needed for layout resolution, avoiding table lookups entirely.
- **Charset inherited from the database.** No table-level charset/collation declarations; columns storing non-ASCII payloads (JSON, `pattern_raw`) rely on the database default.
- **CDC consistency exclusions.** JSON payload columns (`default_record`, `imported`, `authored`) and BLOB content are marked for exclusion from CDC consistency checks — their canonical shape is application-defined, not schema-defined.

Auteur Service API

Auteur Service API

Minimal set of methods organized by concern. All methods use Protobuf over gRPC internally; the Chameleon Fusion UI calls them through the existing Camopress API gateway.

1. Page Resolution

Two resolution methods with different optimization targets.

ResolvePage

Optimized for **performance**. Used by the sync workflow and Content Service to produce resolved page data for end-user rendering. No origin annotations, no change merging — returns the fastest possible result from published data.

None

```
ResolvePage(uri, locale) → ResolvedPageData
```

- Resolves layout overrides for the URI into an ordered list of block IDs (single covering-index query).
- Fetches blocks by IDs + locale.
- Evaluates pre-data rules (redirect, not_found) — returns early if matched.
- Collects and deduplicates all block queries, executes them in parallel.
- Merges dataset fields (authored → imported → default).
- Evaluates post-data rules.
- Maps data to rendering models.
- Returns an ordered list of populated rendering-model instances.

Response:

None

```
ResolvedPageData {  
    status: content | redirect | not_found  
    redirect_url: string (if redirect)
```

```

    redirect_code: int (if redirect)

    blocks: []{position, model_name, model_data}
}

```

ResolveChanges

Optimized for **flexibility**. Used by Chameleon Fusion for preview. Accepts a changeSet object, merges it with published data, and annotates each block with origin.

None

```
ResolveChanges(uri, locale, changeset) → PreviewPageData
```

- Fetches published layout overrides for the URI.
- Merges changeset layout edits (add/remove/modify overrides).
- Resolves the merged layout into block IDs.
- Fetches published blocks, then overlays changeset block edits.
- Evaluates rules and queries datasets (same as ResolvePage).
- Annotates each block with origin: source level (e.g., "inherited from /region"), whether it was modified in the changeset, and which fields changed.
- Returns resolved page data with annotations.

Response:

None

```

PreviewPageData {

    status: content | redirect | not_found

    redirect_url: string (if redirect)

    redirect_code: int (if redirect)

    blocks: []{position, model_name, model_data, origin}

    origin: {

        source_pattern: string    -- which layout override provided this block

        modified_in_changeset: bool
    }
}

```

```
    changed_fields: []string
  }
}
```

2. Layout Overrides

CRUD scoped by namespace. Authors must have Editor permission on the namespace.

None

```
ListLayoutOverrides(domain, namespace) → []{id, pattern, position, weight,
block_id}

GetLayoutOverride(id) → LayoutOverride

CreateLayoutOverride(domain, namespace, pattern, position, weight, block_id) →
id

UpdateLayoutOverride(id, weight?, block_id?) → LayoutOverride

DeleteLayoutOverride(id)
```

3. Blocks

CRUD scoped by block ID and locale.

None

```
ListBlocks(namespace, locale?, page, page_size) → []{id, locale, version,
model_name}

GetBlock(id, locale) → Block

CreateBlock(locale, query_resource?, query_key?, content) → id

UpdateBlock(id, locale, query_resource?, query_key?, content?) → Block

DeleteBlock(id, locale)

GetBlockVersions(id, locale) → []{version, updated_at, updated_by}

GetBlockVersion(id, locale, version) → Block
```


4. Datasets

CRUD for dataset definitions and individual rows.

Dataset definitions

None

```
ListDatasets(page, page_size) → []{id, name, primary_key, update_frequency}

GetDataset(id) → Dataset

CreateDataset(name, primary_key, secondary_index?, default_record?,
              querybuilder_link?, update_frequency?, localized_fields?) → id

UpdateDataset(id, default_record?, querybuilder_link?, update_frequency?,
              localized_fields?) → Dataset

DeleteDataset(id)
```

Dataset rows

None

```
GetRow(dataset_id, row_key, locale?) → {row_key, imported, authored, merged}

ListRows(dataset_id, locale?, page, page_size) → []{row_key, imported,
authored}

UpdateRow(dataset_id, row_key, locale?, authored) → Row

DeleteRow(dataset_id, row_key)                -- soft delete

RestoreRow(dataset_id, row_key)                -- undo soft delete

ImportRows(dataset_id, csv_file)              -- bulk import to authored column

ExportRows(dataset_id, locale?) → csv_file
```

Dataset sync (internal, triggered by scheduler)

None

```
TriggerSync(dataset_id)                      -- execute QueryBuilder SQL, apply
diff

GetSyncStatus(dataset_id) → {last_sync, generation, rows_added, rows_updated,
```

```
rows_removed}
```

5. ChangeSets

Group draft edits for atomic preview and publish.

None

```
CreateChangeSet() → id

GetChangeSet(id) → ChangeSet

AddBlockEdit(changeset_id, block_id, locale, content) → ChangeSet

AddLayoutEdit(changeset_id, domain, pattern, position, namespace, weight,
block_id) → ChangeSet

RemoveEdit(changeset_id, edit_id) → ChangeSet

PublishChangeSet(changeset_id)          -- copies edits to published
tables atomically

DiscardChangeSet(changeset_id)

ListChangeSets(author?, status?, page, page_size) → []{id, status, created_by,
created_at}
```

6. Namespaces & Permissions

None

```
ListNamespaces(domain) → []{namespace, domain}

CreateNamespace(domain, namespace) → Namespace

DeleteNamespace(domain, namespace)

ListPermissions(domain, namespace) → []{user_id, role}

GrantPermission(domain, namespace, user_id, role)    -- role: viewer | editor |
admin

RevokePermission(domain, namespace, user_id)
```

Method Summary

Group	Method	Called by	Optimized for
Resolution	ResolvePage	Sync workflow	Performance — published data only
Resolution	ResolveChanges	Chameleon Fusion	Flexibility — merges changeset
Layout	CRUD	Chameleon Fusion	Authoring
Blocks	CRUD + versions	Chameleon Fusion	Authoring
Datasets	CRUD + rows + sync	Chameleon Fusion / Scheduler	Authoring / Pipeline
ChangeSets	Draft → publish flow	Chameleon Fusion	Change management
Namespaces	Permissions	Chameleon Fusion / Admin	Access control

Presentation Slides

Presentation Slides

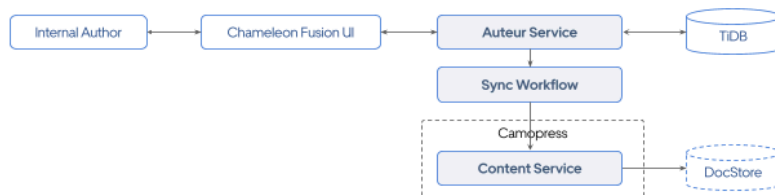
The Goal

1. Enable SEO pages Authoring via Auteur (backend)

Recently, we implemented content rendering of **webeats SEO pages** with Go templates sourcing content from **DocStore**.



Proposal: we are going to implement content Authoring for the SEO pages and synchronization workflow.



The Strategy

2. New service to address current architecture antipatterns

Current State: Fragmented Services

Multiple services communicate through Thrift/Protobuf: Typewriter, Unified-Content, Guac, Louvre, Eats-SEO-Gateway, Camopress.

- Each performs multiple **sequential queries** for data fetching, leading to higher latency.
- **~20% of code** is dedicated solely to mapping data to and from Thrift/Protobuf.
- **~30% of code** implements functionality already available out-of-the-box in relational databases.
- Confusing user experience.

Strategy

Consolidate Authoring into Auteur

- Merge capabilities of Typewriter, Unified-Content, Camopress, and Louvre in Auteur
- Minimize sequential lookups leveraging relational database
- Preserve existing Camopress features for existing pages while adding new functionality for new SEO pages

The Product

3. Content Management System: URL → Layout as set of Blocks

Since we are building new service, let's define desired capabilities.

Hierarchical layout

A base layout with block-level overrides at nested levels (country, vertical, slug).

Dynamic data mapping

Blocks can fetch and embed data from datasets, e.g. region, city, product.

Dataset management

Schema, record editor, import/export, preview.

Localization

Translated and locale-specific content for each locale.

Change management

Draft versions and a history of published changes.

The Proposal

4. Hierarchical Layout: URL → Set of Blocks

A layout for a page defines an ordered list of blocks.

Hierarchical layout: A base layout with block-level overrides at nested levels (country, slug).

The layout can be thought of as a three-dimensional override structure with axes:

- URL pattern (e.g., `ubereats.com/region/*/something`)
- block position
- weight

At retrieval time, the engine filters overrides by the URL pattern, groups them by position, and selects the highest-weight override for each position.

Layout resolution for `ubereats.com/region/san-francisco`

URL level	Blocks				
ubereats.com	Header	Hero	Categories	SEO Text	Footer
/region		Region Hero		Region SEO	
/san-francisco			SF Categories	~~hidden~~	SF Footer

Header	Region Hero	SF Categories	~~hidden~~	SF Footer
--------	-------------	---------------	------------	-----------

The Proposal

5. Dynamic data mapping

Blocks

A block is not a template — it is a data mapping from datasets to an EatsWebSEO model. Each block uses structured fields (queries, exceptions, content) instead of unstructured uScript.

Because queries are declared separately from the mapping logic, the engine can collect all queries across all blocks on a page and execute them in parallel. This eliminates the sequential chained lookups in uScript (e.g., region → city → products).

Structured fields

1. Queries: decoupled, plural, named
2. Exceptions: redirect, not found, hide
3. Exports: EatsWebSEO model, explicit fields

```
queries:
  region:
    source: rolodex
    method: rolodexLookup
    params:
      code: $path.3 # slug

exceptions:
  - when: "= region == nil"
    return: redirect
    redirect_url: "= some_params"
    redirect_code: 301
    reason: "Redirect to another page"
  - when: "= region.active == false"
    return: null
    reason: "Region is inactive"

exports:
  Name: "region.name"
  City: "region.city"
```

The Proposal

6. Dataset management

Content of our pages needs two fundamentally different types of data:

- **Editorial content** — airport descriptions, FAQ answers, "how to request a ride" steps, localized city names.
This content is written by humans, translated per locale.
- **Operational data** — ride ratings, trip counts, pricing, city coordinates, product availability.
This data is derived from production systems, covers millions of entities

Editorial content (currently Typewriter):

- Schema based, JSON values
- Small size: up to 10K records
- Managed by Authors through UI
- Localized to a set of predefined locales

Operational data (currently Rolodex):

- Schema based, JSON values
- Large size: up to 1M records
- Managed by automatic pipeline from Hive
- Not localized

The Proposal

7. Custom Localization

Default Localization

Services load a translation snapshot into memory at startup and perform fast in-memory lookups at render time.

This works well when:

- The number of strings is small and known at build time.
- The service is tolerant for translation distribution delay.
- The source content is mostly permanent.
- The translations do not need overrides.

Our CMS breaks all these assumptions.

Custom Localization

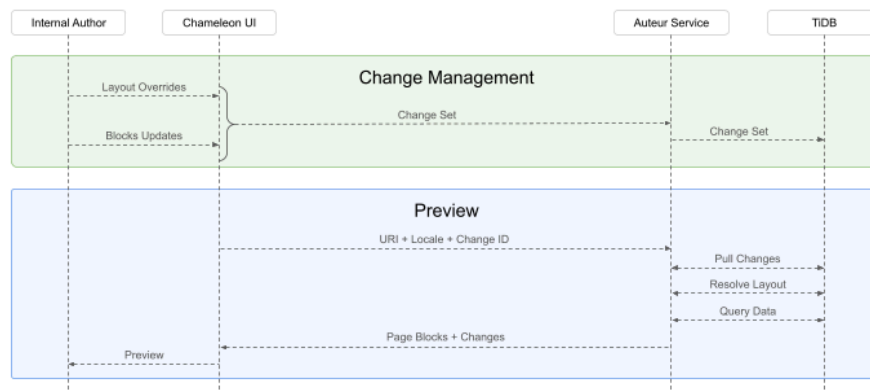
- Store translations in TiDB next to the content
- Retrieve only what the requested page needs
- Integrate with MT-service directly for translations.
- Translate block content and editorial datasets.
- Allows translation overrides.

The Proposal

8. Change management

Authors create or change content (layouts, blocks, datasets) through Chameleon UI.

Our system combines the changes, stores them, shows preview, and publishes the changes.

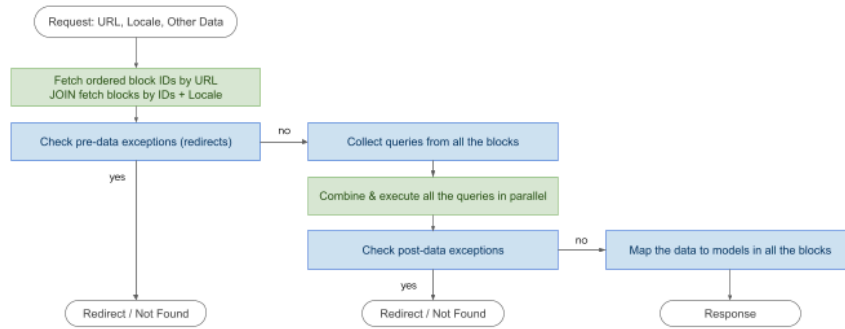


The Proposal

9. Rendering Flow

The diagram illustrates data fetching in two sequential phases:

first resolve the layout into an ordered list of blocks, then execute all block queries in parallel.



Structured Blocks

Structured Blocks

Rationale

In the current architecture, a uScript template fetches data and produces rendered output in the same step. This coupling creates two problems:

1. **Sequential data fetching.** Each block fetches its own data inline — airport code → airport record → territory ID → city → rides. The next lookup can't start until the previous one returns. A single airport page makes 30+ sequential calls.
2. **No cross-block optimization.** Because data fetching is embedded in template logic, the engine can't see what data a block needs until it starts executing. Two blocks that need the same airport record fetch it independently.

Blocks in the new system are **data mappers, not templates**. A block does not produce HTML or any rendered output. It describes:

- What data to fetch (queries).
- When to abort (exceptions).
- How to map the data to a typed structure (content).

By separating data declarations from processing, the engine can:

- Collect all queries from all blocks on a page before executing any of them.
- Deduplicate identical queries (same dataset + same key = single fetch).
- Execute all queries in parallel.
- Evaluate block logic only after all data is available.

Blocks in this document are represented as YAML for readability only. Authors edit blocks through structured form fields in the Chameleon UI — they do not interact with YAML directly. Internally, blocks are transmitted to the backend and stored in TiDB as Protobuf to minimize decoding time at resolve.

Structure

A block has six top-level fields:

None

`model: RenderingModelName`

`# required — which Go struct this block`

```

maps to
description: "Human-readable" # optional – for authoring UI

queries: # optional – named data source lookups
  name:
    dataset: dataset_name
    key: expr # expression evaluated against $path,
$locale, $geo
    index: secondary # optional – use secondary index
instead of primary key

vars: # optional – computed intermediate
values
  name: expr

exceptions: # optional – abort conditions
  - when: expr # boolean expression
    return: redirect | not_found | null
    redirect_url: expr # if redirect
    redirect_code: 301 | 302 # if redirect
    reason: "string" # for logging and preview annotations

content: # required – field mappings to the
rendering model
  fields:
    FieldName: expr
    NestedField: expr # expr can return maps/arrays for
nested models

```

Field descriptions

model — the name of the rendering model (Go struct) registered in EatsWebSEO that this block maps data into. The engine validates content field names and types against the model schema at publish time. Moving **model** to the top level (rather than nesting it under **content**) enables validation without parsing content expressions.

description — a human-readable label displayed in the Chameleon authoring UI. Helps authors browse, search, and understand blocks without reading the expressions.

queries — named data source declarations. Each query specifies a **dataset** name and a **key** expression that resolves to a lookup key. The key expression can reference URL path segments (**\$path.3**), the current locale (**\$locale**), geo data (**\$geo.country**), or static values. All queries across all blocks on a page are collected, deduplicated, and

executed in parallel by the engine.

vars — computed intermediate values, evaluated once after queries resolve. Vars can reference query results and other vars (in declaration order). They become part of the expression environment for exceptions and content. Vars eliminate repeated expressions — the equivalent of `setGlobal` in uScript, but scoped to the block.

exceptions — an ordered list of conditions evaluated top-down. The first matching exception determines the block result. If no exception matches, the block proceeds to content mapping. Exception types:

- **not_found** — the page does not exist for this URL (applies to the entire page).
- **redirect** — the page should redirect (applies to the entire page).
- **null** — this block has nothing to show (block is hidden, page continues).

content — field mappings from the expression environment (query results + vars) to the rendering model. Each field value is an expr expression. Expressions can return scalars, maps (for nested models), or arrays of maps (for repeated nested models).

Evaluation order

None

`queries → vars → exceptions → content`

Each stage can reference outputs from all previous stages. The engine resolves dataset fields (authored → imported → default) before passing data to vars/exceptions/content — blocks never see the raw imported/authored split.

Expression language

All expressions in **vars**, **exceptions.when**, **exceptions.redirect_url**, and **content.fields** use [Expr](#). Key capabilities:

- Field access: `airport.name`, `airport.gates[0].terminal`
- Optional chaining: `airport?.city?.name`
- Nil coalescing: `airport.shortName ?? "Unknown"`
- Conditionals: `airport.active ? "Open" : "Closed"`
- String functions: `trim()`, `upper()`, `lower()`, `replace()`, `split()`
- Pipe operator: `airport.name | trim() | upper()`
- Array operations: `map()`, `filter()`, `sortBy()`, `len()`
- Map construction: `{Key: value, Other: value}`

Expressions are compiled at publish time and stored as bytecode. At resolve time, only bytecode evaluation occurs — no parsing.

Expression environment

Every expression has access to:

Variable	Source	Example
\$path	URL segments (after preprocessing)	\$path.3 → "lax"
\$locale	Resolved locale	"en", "fr"
\$geo	Geo IP data	\$geo.country → "US"
\$requestUrl	Full request URL	For canonical URL comparison
Query results	Named by query key	airport.name, city.lat
Vars	Named by var key	shortName, showBusiness

Examples

Example 1: Airport Hero Block

A block that queries airport and city data, computes display values, handles missing data, and maps to the **AirportHeroBlock** rendering model.

None

```
model: AirportHeroBlock
description: "Airport page hero section with name, city, and ride info"

queries:
  airport:
    dataset: airports
    key: $path.3 | upper()
  city:
    dataset: cities
    key: airport.territoryId
    index: secondary

vars:
```

```

shortName: airport.shortName | trim()
name: airport.name | trim()
code: airport.code | upper()
heroBody: |
  airport.pageHeroBody
  | trim()
  | replace("${shortName}", shortName)
  | replace("${name}", name)
  | replace("${airportCode}", code)

exceptions:
- when: airport == nil
  return: not_found
  reason: "Airport not found for code"
- when: airport.active == false
  return: null
  reason: "Airport is inactive"

export:
  fields:
    Name: name
    ShortName: shortName
    Code: code
    City: city?.name ?? ""
    Latitude: city?.lat
    Longitude: city?.lng
    HeroBody: heroBody
    ShowBusiness: airport.businessPercent >= 0.12

```

What this replaces: ~100 lines of the airport uScript that extract airport names, look up city centroids via territory ID, compute display strings, and set globals.

Example 2: Airport FAQ Block

A block on the same airport page that maps FAQ data. Shares the same **airports** dataset query — the engine deduplicates it with Example 1.

None

```

model: AirportFAQBlock
description: "Airport page FAQ section"

queries:
  airport:

```

```

    dataset: airports
    key: $path.3 | upper()

vars:
  shortName: airport.shortName | trim()
  code: airport.code | upper()

exceptions:
  - when: airport == nil
    return: null
    reason: "No airport data for FAQ"
  - when: airport.faqs == nil or len(airport.faqs) == 0
    return: null
    reason: "No FAQs available"

content:
  fields:
    FAQs: |
      airport.faqs | map(faq, {
        Question: faq.question
          | replace("${airportCode}", code)
          | replace("${shortName}", shortName),
        Answer: faq.answer
          | trim()
          | replace("${shortName}", shortName)
      })

```

What this replaces: ~50 lines of the airport uScript that manually index `faqs[0]` through `faqs[3]`, extract question and answer, and do string replacements on each.

Example 3: Airport Terminals Block

Demonstrates array mapping with nested fields, replacing the 500+ lines of manually unrolled terminal iteration in the existing uScript.

None

```

model: AirportTerminalsBlock
description: "Airport terminals with restaurants, bars, lounges, shops"

queries:
  airport:
    dataset: airports
    key: $path.3 | upper()

```



```

exceptions:
  - when: airport == nil or airport.terminals == nil
    return: null
    reason: "No terminal data"

content:
  fields:
    Terminals: |
      airport.terminals | map(t, {
        Name: t.name | trim(),
        Restaurants: t.restaurants | trim(),
        Bars: t.bars | trim(),
        Lounges: t.lounges | trim(),
        Shops: t.shops | trim()
      })

```

What this replaces: ~500 lines of the airport uScript that manually iterate `terminals[0]` through `terminals[11]`, each with 6 fields and try/catch wrappers.

Example 4: Store Page Block

A block for a store (restaurant) page with redirect logic for canonical URLs.

None

```

model: StoreBlock
description: "Store page with name, address, category, and menu link"

queries:
  store:
    dataset: stores
    key: $path.3

exceptions:
  - when: store == nil
    return: not_found
    reason: "Store not found"
  - when: store.canonicalUrl != nil and store.canonicalUrl !=
$requestUrl
    return: redirect
    redirect_url: store.canonicalUrl
    redirect_code: 301
    reason: "Redirect to canonical store URL"

```

```

- when: store.active == false
  return: null
  reason: "Store is inactive"

content:
  fields:
    Name: store.name
    Address: store.address
    Category: store.category
    ImageUrl: store.heroImageUrl ?? ""
    Rating: store.avgRating
    MenuUrl: store.menuUrl
    IsNew: store.createdDaysAgo < 30

```

Example 5: Region Page Block

A block for a region landing page with multiple queries — region data and a list of top cities in that region.

None

```

model: RegionBlock
description: "Region page with description, top cities, and ride
products"

queries:
  region:
    dataset: regions
    key: $path.2
  topCities:
    dataset: region_top_cities
    key: $path.2
    index: secondary
  products:
    dataset: ride_products
    key: region.territoryId
    index: secondary

vars:
  regionName: region.name | trim()

exceptions:
- when: region == nil
  return: not_found
  reason: "Region not found"

```

```

- when: region.active == false
  return: null
  reason: "Region is inactive"

content:
  fields:
    Name: regionName
    Description: |
      region.description
      | replace("${regionName}", regionName)
    Country: region.country
    Cities: |
      topCities | filter(c, c.active) | sortBy(c, c.population) |
map(c, {
  Name: c.name,
  Slug: c.slug,
  Population: c.population
})
  Products: |
    products | filter(p, p.available) | map(p, {
      Name: p.displayName,
      Description: p.description ?? "",
      MaxCapacity: p.maxCapacity
    })

```

Three queries, all executed in parallel. Note that `products` references `region.territoryId` — this is a cross-query dependency. The engine detects this and executes `region` first, then `topCities` and `products` in parallel once `region` resolves. Authors don't manage execution order; the engine infers it from expressions.

Example 6: Queryless Block — Pure Redirect

A block with no queries that redirects based on URL structure. Evaluated before any data fetching occurs.

None

```

model: RedirectBlock
description: "Redirect old country-prefix URLs to canonical format"

exceptions:
- when: $path.1 == "us" and $path.2 != nil
  return: redirect

```

```

      redirect_url: "'/' + $path.2 + '/' + ($path.3 ?? '')"
      redirect_code: 301
      reason: "Redirect /us/city/... to /city/..."
    - when: $path.1 == "gb" and $path.2 != nil
      return: redirect
      redirect_url: "'/' + $path.2 + '/' + ($path.3 ?? '')"
      redirect_code: 301
      reason: "Redirect /gb/city/... to /city/..."

content:
  fields: {}

```

No queries, no vars, no content. The engine detects that this block has no queries and evaluates it before fetching any data. If the exception matches, the entire page redirects immediately — no other blocks are evaluated.

Example 7: Static Content Block

A block with no queries that provides static content from inline values. Useful for headers, footers, or promotional banners that don't depend on data.

None

```

model: PromoBannerBlock
description: "Promotional banner for first-time riders"

vars:
  showPromo: $geo.country in ["US", "CA", "GB", "AU"]

exceptions:
  - when: not showPromo
    return: null
    reason: "Promo not available in this country"

content:
  fields:
    Headline: "'Get 50% off your first ride'"
    Body: "'Download the Uber app and use code WELCOME50 at checkout.'"
    PromoCode: "'WELCOME50'"
    CtaText: "'Sign up now'"
    CtaUrl: "'https://www.uber.com/signup'"

```

No queries, no dataset dependency. All field values are literal strings (quoted in expr).

The `vars` section uses `$geo.country` from the request context to conditionally hide the block in unsupported countries.

Example 8: Airport Rides Block — Multiple Queries with Computed Display

A block that combines airport data with ride product information and trip statistics.

None

```
model: AirportRidesBlock
description: "Available ride products at the airport with ratings and pricing"

queries:
  airport:
    dataset: airports
    key: $path.3 | upper()
  rideNames:
    dataset: ride_products
    key: airport.territoryId
    index: secondary
  tripData:
    dataset: route_trip_data
    key: airport.routeSlug

vars:
  cityId: airport.territoryId

exceptions:
  - when: airport == nil
    return: null
    reason: "No airport data"
  - when: rideNames == nil
    return: null
    reason: "No ride products for this airport"

content:
  fields:
    Products: |
      rideNames | filter(r, r.displayName != nil) | map(r, {
        Name: r.displayName,
        Description: r.description ?? "Affordable rides",
        MaxCapacity: r.maxCapacity,
        Rating: tripData?.ratings[r.productType] ?? nil,
        Price: tripData?.prices[r.productType] ?? nil
      })
    OverallRating: tripData?.overallAvgRating
```

```
TotalRatings: tripData?.totalRatings
```

Three queries with dependency chain: `rideNames` and `tripData` both depend on fields from `airport` (`territoryId`, `routeSlug`). The engine executes `airport` first, then `rideNames` and `tripData` in parallel.

How the engine handles cross-block query deduplication

When a page has multiple blocks that query the same dataset with the same key, the engine fetches the data once:

None

```
Page: /airport/lax (locale: en)
```

```
Block 1 (AirportHeroBlock):  queries.airport → dataset: airports,  
key: "LAX"
```

```
Block 2 (AirportFAQBlock):   queries.airport → dataset: airports,  
key: "LAX"
```

```
Block 3 (AirportTerminals):  queries.airport → dataset: airports,  
key: "LAX"
```

```
Block 4 (AirportRidesBlock): queries.airport → dataset: airports,  
key: "LAX"
```

```
                                queries.rideNames → dataset:  
ride_products, key: <airport.territoryId>  
                                queries.tripData → dataset:  
route_trip_data, key: <airport.routeSlug>
```

Engine execution:

Phase 1: fetch airports[LAX] – one query, result shared by all 4 blocks

Phase 2: fetch ride_products[territoryId], route_trip_data[routeSlug] – in parallel

Phase 3: evaluate all blocks with resolved data

Four blocks, three unique queries, two phases. The existing uScript makes 30+ sequential calls for the same data.

Query Complexity Limits

Risk: deep query chains

Nothing in the block structure inherently prevents an author from creating deeply nested query dependencies:

None

```
queries:
  airport:
    dataset: airports
    key: $path.3 | upper()
  city:
    dataset: cities
    key: airport.territoryId
  country:
    dataset: countries
    key: city.countryCode
  regulations:
    dataset: transport_regulations
    key: country.regulationId
  products:
    dataset: ride_products
    key: regulations.productGroupId
```

Five queries, four levels of dependency. Each level adds a sequential phase — either a roundtrip to TiDB or a CTE join that becomes progressively harder to optimize. This is exactly the problem the new architecture set out to solve: reimplementing SQL JOINS imperatively, just with cleaner syntax.

Constraint: maximum dependency depth of 1

The engine enforces a hard limit of **one level of query dependency**. A query's key expression can reference URL/locale/geo context or a direct result of another query, but not a result of a query that itself depends on another query.

None

```
Level 0: key uses $path, $locale, $geo, or static values → always allowed
Level 1: key uses a Level-0 query result → allowed
Level 2+: key uses a Level-1+ query result → rejected
at publish time
```

This guarantees at most two-phase query execution: all Level-0 queries in parallel, then all Level-1 queries in parallel. The engine can further optimize Level-0 → Level-1 chains into single SQL joins, reducing many pages to one phase.

If an author needs data that is two hops away, the data should be pre-joined in the dataset at pipeline time. The pipeline can execute `airport JOIN city JOIN country` once per import cycle; the block should not redo it on every page request.

Additional soft limits

Constraint	Value	Enforced at	Type
Max dependency depth	1	Publish time	Hard error — block cannot be published
Max queries per block	4	Publish time	Warning — author can override with justification
Max total unique queries per page	16	Publish time	Warning — suggests dataset denormalization

If a block needs more than 4 queries, it is likely doing too much. Splitting it into multiple blocks is preferred — they share query results through cross-block deduplication anyway.

These limits provide a performance guarantee: **every page resolves in at most three phases** (layout resolution → Level-0 queries → Level-1 queries), and the engine can optimize most pages down to two.

Alternative: uScript Optimization

Alternative: LLM-Assisted uScript Optimization

What the proposal actually is

Two moves, applied to the existing stack (uScript in Camopress + Louvre blocks + Rolodex/Typewriter datasources):

1. **Batch-migration.** Train a code-mode skill that ingests an existing uScript, identifies chained lookups (`typewriterInstance` → field → `rolodexSecondaryLookup` → field → next call), and rewrites them to use batch/multi-key APIs where available, hoisting independent lookups above the first `setGlobal` chain so they can be parallelized.
2. **Ongoing guardrail.** Attach an LLM reviewer to the uScript editing flow. On save/publish, it inspects the new script for anti-patterns — deep call chains, N+1 loops over `typewriterInstance`, `try/else` swallowing errors — and blocks or comments PRs before they land in Camopress.

What this actually delivers

Problem in the current stack	Does this alternative solve it?
30+ sequential calls per page	Partially — only where batch APIs already exist and dependencies allow reordering. The airport script has genuine chains (airport_code → airportTW.territoryId → cityLookup) where no reordering helps without an engine change.
Fragmented authoring surface (Typewriter, Unified-Content, Louvre, Camopress)	No. Authors still hop between four tools.
No cross-block deduplication	No. Louvre still hydrates each block independently — the engine has no visibility into "block A and block B both fetch LAX."
Layout / hierarchical overrides in a single query	No. Still handled ad-hoc per page type.
Change tracking ("what pages does this edit affect?")	No. No structural model of the content graph exists to walk.
Localization at scale	No. Rosetta + Typewriter split remains.
Editorial vs. imported data separation with soft-delete	No. Typewriter row lifecycle unchanged.
Preview with provenance	No. Requires the resolver to know which override provided each

Problem in the current stack	Does this alternative solve it?
	block.

The alternative is a **latency-optimization project**, not a CMS project. That's a fair choice, but it's a much narrower one.

Where the alternative genuinely wins

- **Zero migration risk.** No shadow-mode diffing across two systems, no cutover, no dual-write. Every uScript rewrite ships one page type at a time; if it regresses, revert the template.
- **No new service to operate.** No TiDB dependency, no sync workflow, no changeset engine, no `ResolvePage/ResolveChanges` split to maintain.
- **Ships value in weeks, not quarters.** An LLM skill that batches `typewriterInstance` calls into `typewriterInstanceBatch` (if that exists) or a Louvre-level `getBlocksBatch` can be deployed against airport pages in a sprint. The greenfield ERD is a multi-quarter build before the first page moves.
- **Compounds without a lock-in.** Every optimized uScript is a standalone win. If the greenfield CMS later ships, the optimized scripts still run — they're not wasted work.

Where the alternative can't take you

- **The chain that has to exist.** The airport script does `path[3] → airport_code_upper → typewriterInstance("LAX") → .territoryId → rolodexLookup(territoryId, "cities")`. No amount of code review or batching removes that two-hop dependency; only a **join at the datasource layer** or a **denormalized dataset** does. The alternative has no place to put denormalization — Typewriter and Rolodex are the sources of truth.
- **Cross-block de-dup is architectural.** Even a perfectly-optimized airport uScript still can't tell the FAQ block "we already fetched LAX." That requires a resolver that sees all blocks on a page before executing any of them. uScript doesn't have that layer.
- **LLM reviewer is a soft gate, not a constraint.** Authors will find workarounds (moving logic into included partials, calling `setGlobal` to hide indirection). Enforcement erodes over time unless it's a compiled constraint. Compare to the greenfield's *hard* depth-1 publish-time check — it can't be talked around.
- **You're paying twice.** Ongoing engineering hours to maintain the LLM skill + reviewer + optimized templates, without retiring any of the fragmented services below. That's cost added, not cost removed.

Cost & risk comparison (rough)

	Alternative (optimize uScript)	Greenfield (Auteur)
Time to first latency win	2–4 weeks (one page type)	6+ months (V1 + shadow mode)
Time to fragmented-tooling win	Never	~12–18 months
Engineering headcount to build	1–2 engineers, part-time	4–6 engineers, full-time
Ongoing cost	LLM skill maintenance + template review load	New service on-call, TiDB ownership, sync workflow
Ceiling on latency improvement	~30–50% (bounded by API surface)	~70–90% (bounded by TiDB + parallelism)
Delivers "content team owns launches" outcome	No	Yes
Reversible if wrong	Fully	Requires unwinding a service + a database

The honest read

The alternative is the right proposal **if the actual problem is latency**. It's the wrong proposal if the actual problem is *"we can't launch a page for a new city without three engineers and four tools."* Those are different problems with different solutions, and the ERD is arguing the second is the bigger one.

Hybrid worth considering

Run the LLM optimization work **now** as a bridge, decoupled from the CMS decision:

1. Ship the uScript-batching skill this quarter. Airport and store pages get their p99 cut immediately.
2. Ship the LLM reviewer as a Camopress pre-commit hook. It stops the bleeding on new chained scripts.
3. Use the wins to buy a runway for the greenfield build — you don't have to sell the CMS on latency alone if latency is already improving.
4. When greenfield ships, migrate optimized-uScript pages one by one; the optimized versions have already been mechanically reasoned about, which makes translation to blocks easier.

This turns "either/or" into "sequence." The alternative isn't wasted if the CMS ships; the CMS isn't blocked if the alternative doesn't deliver everything.

The one risk unique to this alternative

An LLM-authored optimizer that rewrites production templates is a very sharp tool. Two failure modes worth naming:

- **Silent semantic drift.** An LLM sees `typewriterInstance("LAX", "airport", "en")` twice and dedupes. But if the second call was inside a `try` that hides errors from a different code path, deduping changes behavior. Every rewrite needs a diff test against production output — same infra as the greenfield's shadow mode, in fact.
- **Reviewer capture.** The bar for "complex enough to reject" drifts as authors and reviewers negotiate exceptions. Six months in, the reviewer approves everything. This is why compile-time constraints beat linters for load-bearing rules.

Both are addressable, but they're the reasons "just add an LLM" is not free.

User Interface

User Interface

We are building the Authoring Platform as a system that resolves a page identified by a URI (e.g., `domain/some/path`) into a layout — an ordered set of authorable blocks with resolved data-retrieval expressions.

Because we want to configure a group of pages at once, configuration is expressed against URI patterns (e.g., `domain/namespace/slot1/*/slot2/...`) rather than individual URIs.

This document walks through an example configuration that maps URI patterns to nested sets of authorable blocks. "Nested" means that blocks configured for a subpath may complement or override blocks configured for a more general path.

Glossary

- **URI** — page identifier, e.g., `www.ubereats.com/category/santa-clara-ca/italian`.
- **Pattern** — a URI-shaped string with some path segments replaced by wildcards, e.g., `www.ubereats.com/category/*/italian`.
- **Namespace** — the first URI path segment after the domain; separates large groups of pages.
- **Changeset** — a pattern plus a set of blocks that authors create, edit, and publish; tracks the history of changes.
- **Dataset** — a named group of data records.

Steps

0. Domain-level blocks.
1. Region-pages blocks.
2. Region-city dataset.
3. Dataset integration with a block.
4. Dishes-pages new scope.
5. Experiments.

Step 0: Header and Footer

Author opens the screen for editing **www.ubereats.com** and sees a few ordered selectors.

The Domain selector is active, other selectors and actions are disabled.

Domain: **www.ubereats.com**

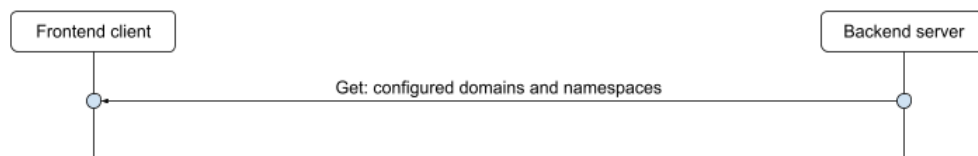
Namespace: **/regions**

Slots: **/ca** **/foo** **/bar** **/baz**

Actions: **Add Block** **Publish**

www.ubereats.com
www.uber.com

Client requests list of domains and namespaces from server.



Author selects domain **www.ubereats.com**.

The system unblocks the **namespace selector** and activates the **Add Block** button.

Let's postpone namespaces and add three static blocks: **Header**, **Main**, and **Footer**.

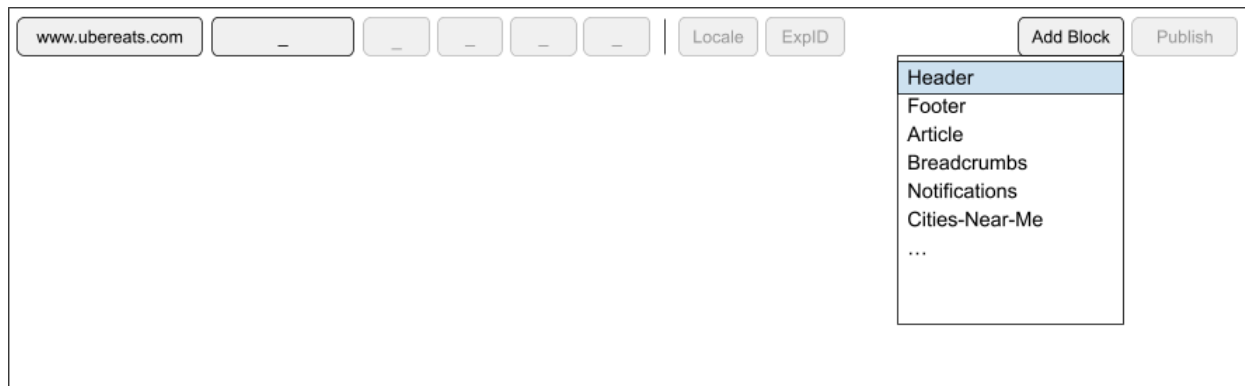
Domain: **www.ubereats.com**

Namespace: **-** **-** **-** **-**

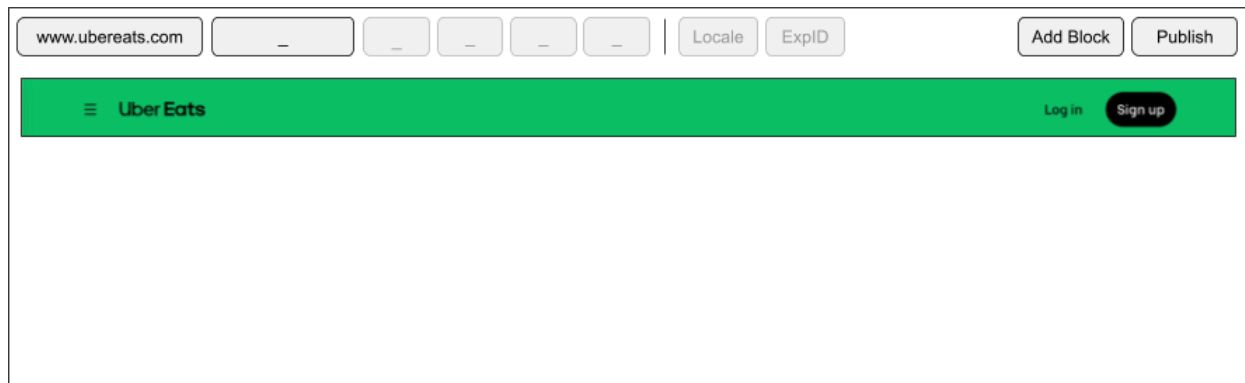
Slots: **-** **-** **-** **-**

Actions: **Add Block** **Publish**

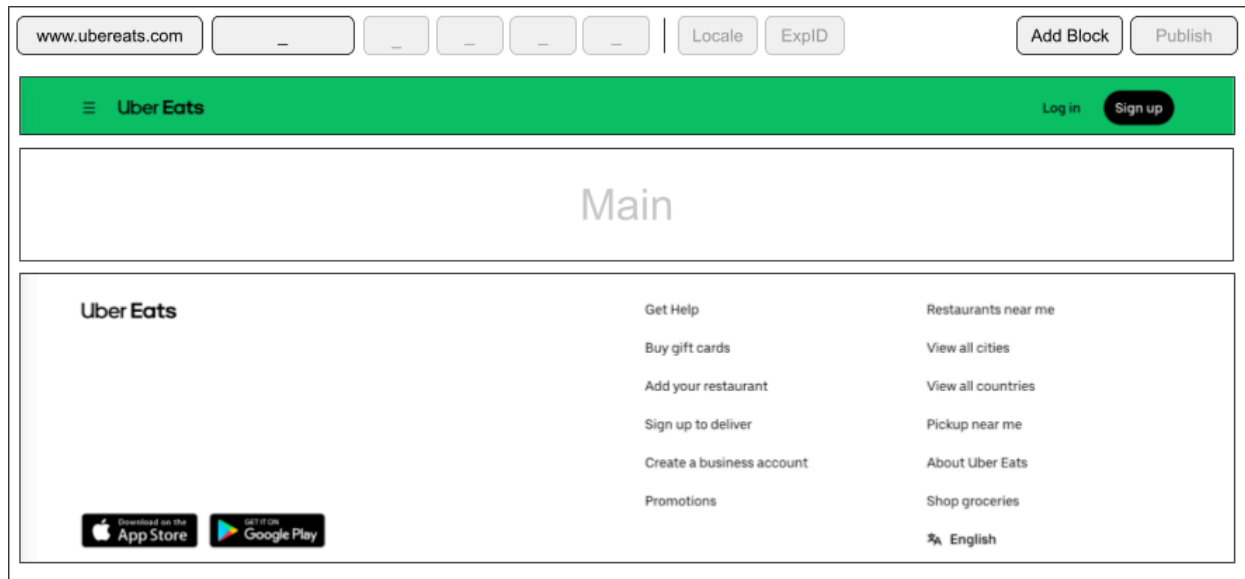
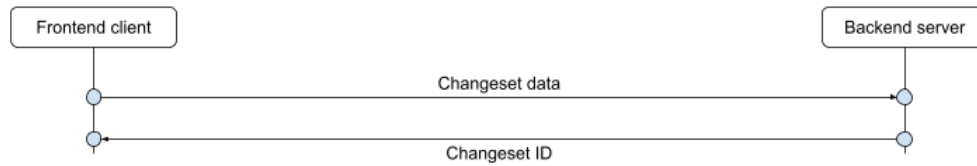
Author clicks on the Add Block button, which opens a list of all existing blocks (from EatsWebSEO schemas).



Author selects Header block, and the system adds the Header with default content values. Frontend requests the backend to create a **Changeset** record with a single block and receives the Changeset ID as response.



The same way Author adds the Main and the Footer block. The Main block is empty (will be hidden in rendering). It is used just to reserve the slot on domain level for other blocks on nested levels. Frontend requests the backend to update the Changeset identified by the ID with other blocks. Now the backend stores the Changeset with three blocks.

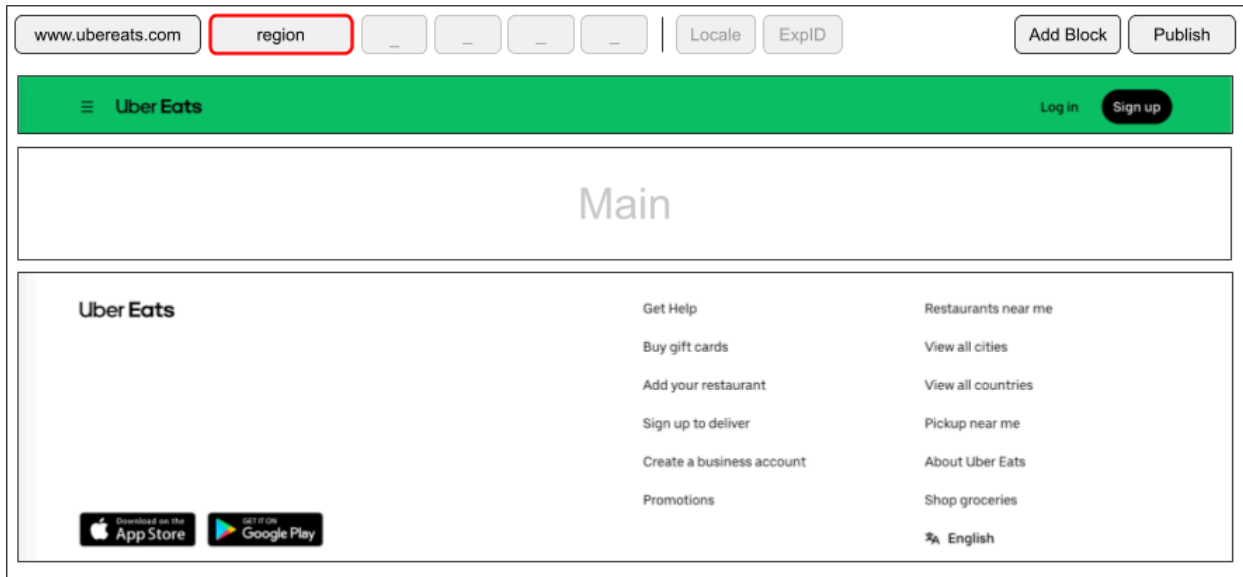


Author publishes the changes.

Now the system will return a page with Header and Footer (hiding empty Main block) when requested www.ubereats.com.

Step 1: Region pages (block override)

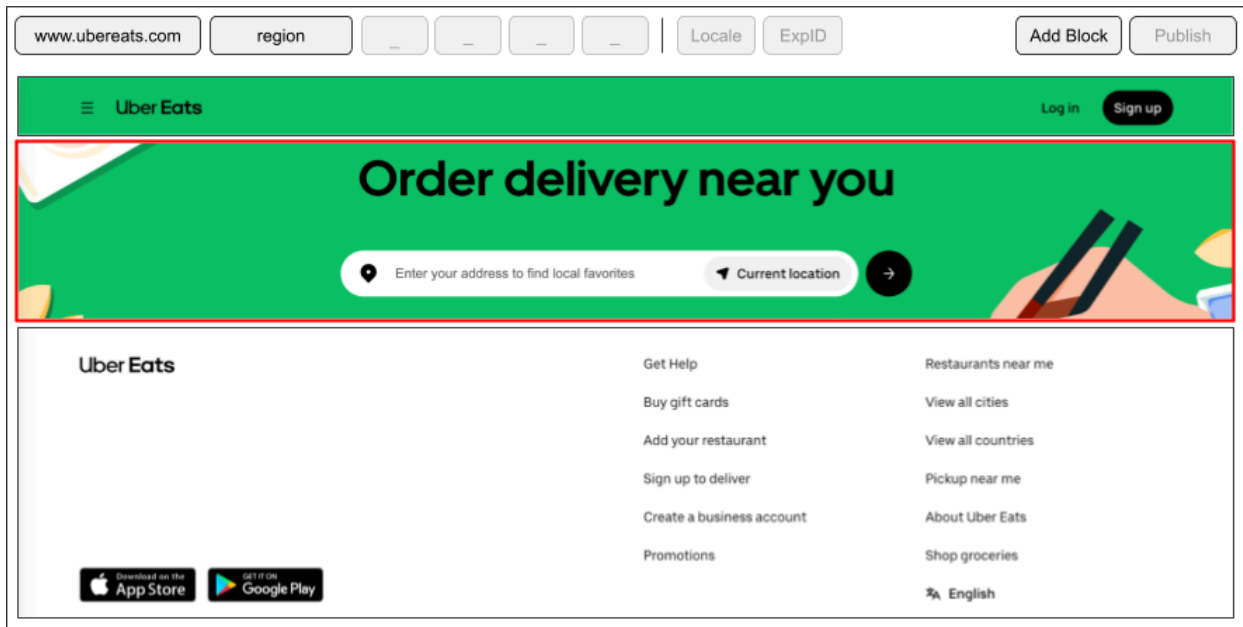
Author creates a new namespace “region”.



Author overrides the Main block with RegionHero block.

Beside the override, Authors have options to insert new blocks before or after any existing blocks.

Frontend requests the backend to create another **Changeset** with a single block and receives the changeset ID in response.



Step 2: Region city dataset

Author creates new dataset with name "region_cities", reference to Querybuilder and default record. Real data will be imported later, the system will use the dataset default record for mocking content.

during block editing.

Create Dataset

Name:

region_cities

Querybuilder ID:

###-###-###-###

Default Record:

```
CA: {
  "header": "Cities in California",
  "items": [
    {
      "name": "San Francisco",
      "id": "san_francisco"
    },
    {
      "name": "San Jose",
      "id": "san_Jose"
    }
  ]
}
```

Step 3: Region data blocks

Author adds the “*” (star placeholder matches any value) as first pattern and adds the Cities block after the RegionHero block

The block appears with stub content. Author needs to wire a dataset with a selector from the request and map fields to block export.

www.ubereats.comregion*

Uber Eats

Order delive

Enter your address to find local favorites

Header

List Item

Uber Eats

Sign up to deliver

Create a business account

Promotions

Pickup near me

About Uber Eats

Shop groceries

English

Download on the App Store

GET IT ON Google Play

Configure Block

Queries

Name: cities

Dataset: region_cities

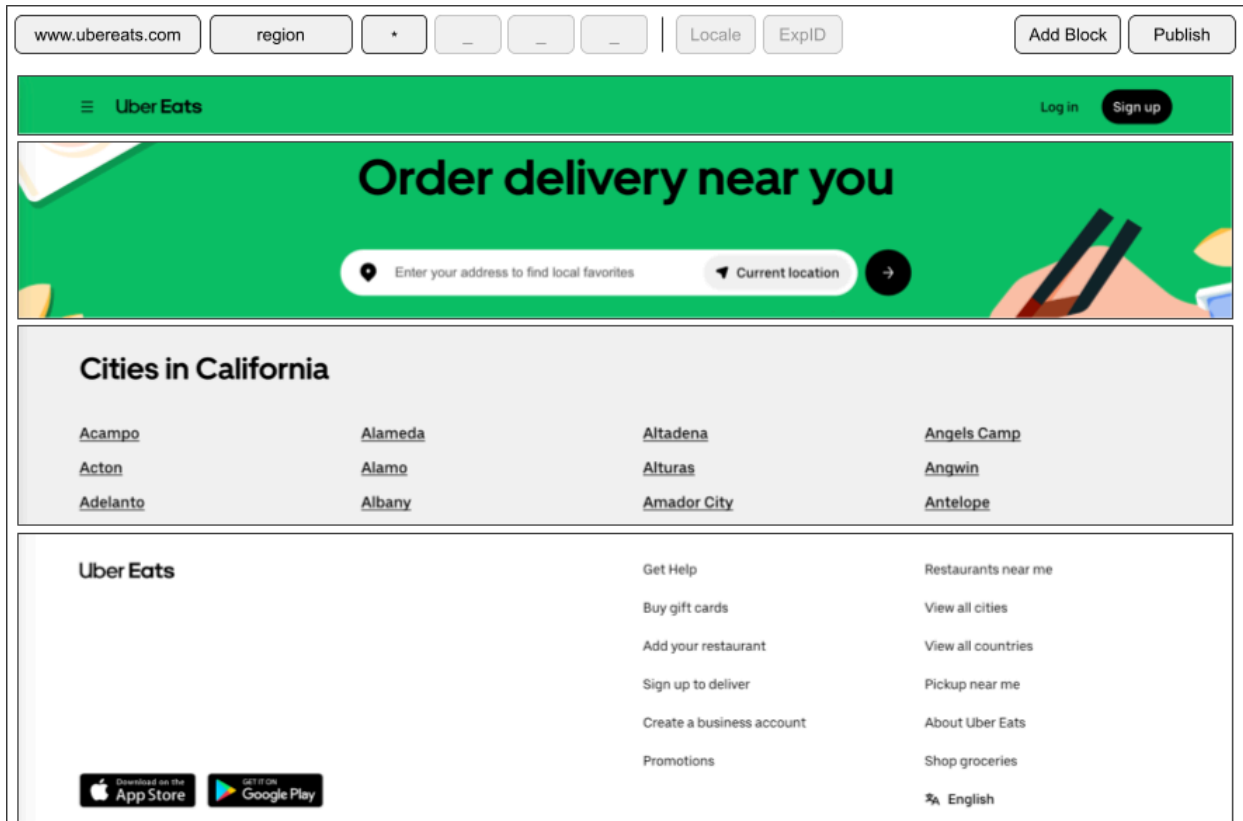
Unique Key: \$env.path[1] // "ca"

Exports

Header: cities.header

Items: cities.items

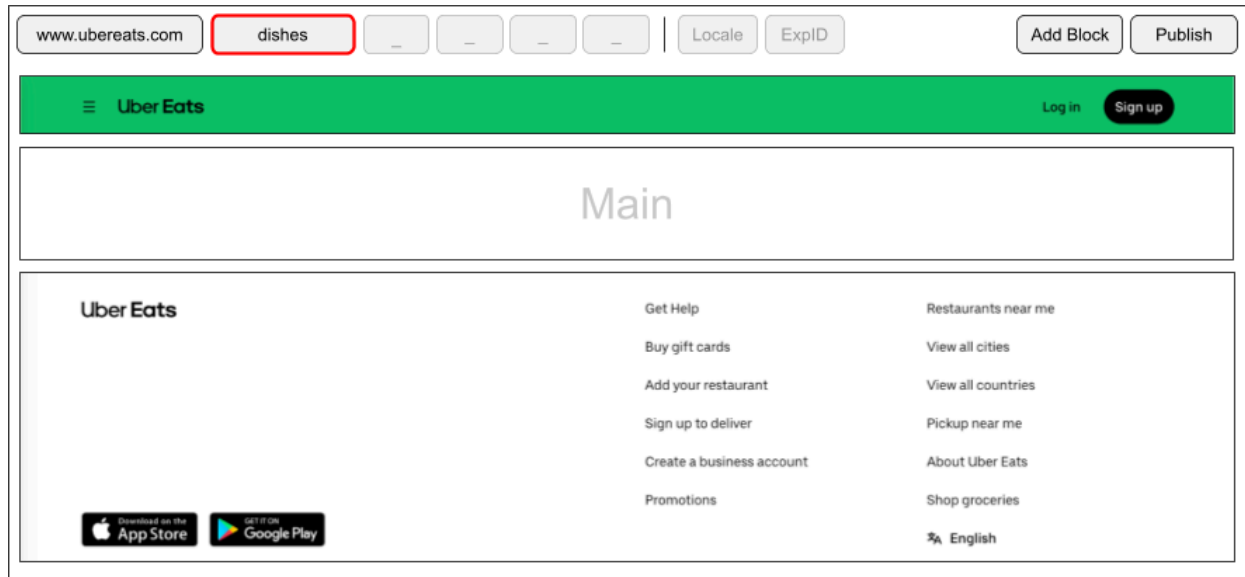
The system uses the dataset default record to mock the content for the Cities block.



Now we have configured pages for all regions included in the region_cities dataset. Header and Footer blocks are inherited from the domain level, Region_Hero block is inherited from namespace level. The Cities block is specific for the first pattern slot.

Step 4: Dishes pages

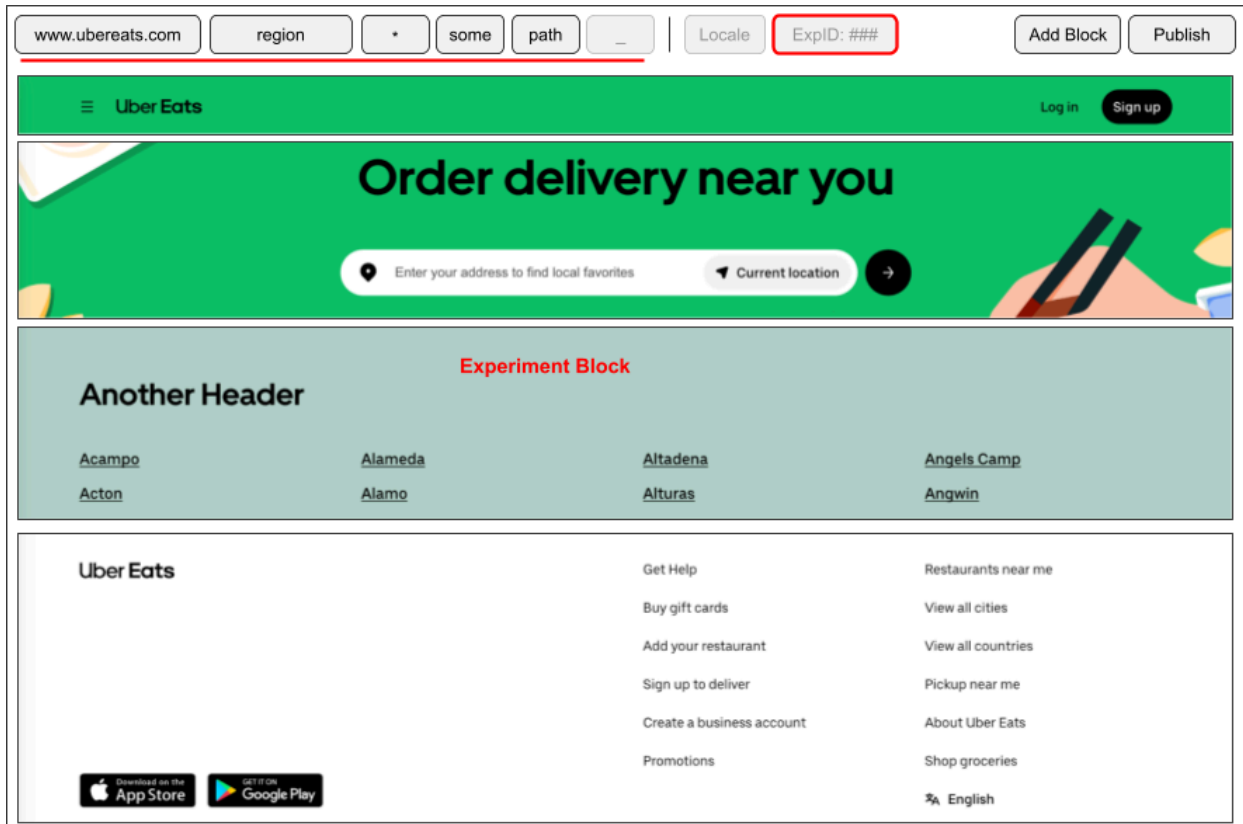
Author creates a new namespace “dishes” and sees three blocks inherited from domain level. Blocks created under the “region” namespace do not appear. Authors can add new blocks or override existing ones.



Experiment Configuration

Author may configure experiments by providing experiment ID, layout override and one or more blocks (before, after, or override).

An experiment block may also refer to a dedicated dataset in a case if we want some specific data.



Metadata

Metadata functions as a standard block within a page layout, though our application must enforce a limit of one metadata block per page. For user interaction, it can be presented as the primary block on the editing interface, within a modal dialog, or on a dedicated screen.

www.ubereats.com

region

*

_

_

_

|

Locale

ExpID

Add Block

Publish

Metadata

Title: foo

Description: bar

☰ Uber Eats

Log inSign up

Order delivery near you

📍 Enter your address to find local favorites

📍 Current location

➔

Cities in California

[Acampo](#)

[Acton](#)

[Adelanto](#)

[Alameda](#)

[Alamo](#)

[Albany](#)

[Altadena](#)

[Alturas](#)

[Amador City](#)

[Angels Camp](#)

[Angwin](#)

[Antelope](#)

Uber Eats

Get Help

Buy gift cards

Add your restaurant

Sign up to deliver

Create a business account

Promotions

Restaurants near me

View all cities

View all countries

Pickup near me

About Uber Eats

Shop groceries

🌐 English

Download on the App Store

GET IT ON Google Play

Authoring Models

Authoring Models

System Requirements

- We are going to design a content management system for tens of millions of pages.
- Sync pipeline will consume page content and store in a caching layer (serving latency is not critical).

Two flows

- authoring: create or modify a group of pages (to address the scale)
- serving: resolve content for an individual page identified by URL

Content model

- A page is an ordered sequence of blocks.
- Block schemas are defined and owned by the rendering engine.
- The CMS maps values from datasets combined with static values to the block fields.

Authoring

- Authors must be able to edit/override content for groups of pages identified by various conditions.
- The content overrides may be applied during the authoring phase (editing) or serving phase (experiments).

Authoring Models Options

Option 1 — Group base + per-page override (current Chameleon)

Two-level model: a group (namespace) defines a base block list; individual pages override specific blocks.

Pros. Simple mental model: $\text{resolved_page} = \text{group_base} \oplus \text{page_overrides}$. Clean change tracking.

Cons. Only two levels. Crosscutting conditions ("all pages in France," "all McDonald's")

don't fit either level. No home for non-URL conditions or serving-time experiments.

Verdict. Insufficient. Any fix evolves into Option 2 or Option 3.

Option 2 — Hierarchical URI-pattern overrides

Overrides authored at multiple URI-pattern levels (domain → namespace → subpath → segment) and composed by pattern specificity.

Pros. Strong deduplication. Fits URL-tree-shaped content well (SEO pages, category trees).

Cons. Non-URL conditions (brand, chain) require a separate mechanism.

Verdict. Viable for URL-shaped overrides; needs bolt-ons for the rest.

Option 3 — Multi-dimensional override records

Each override record is keyed by a set of dimensions — URI pattern, entity attributes (brand, chain, country, category), experiment, locale — with a precedence rule.

Pros. Handles the full override requirement in one primitive. Covers authoring-time overrides and serving-time experiments uniformly.

Cons. Authors reason about dimensions instead of paths — harder UX. Requires resolving non-URL attributes before matching (a two-phase resolution: identity phase → layout phase)